

Unbiased and Second-Order-Free Training for High-Dimensional PDEs

Jaemin Seo¹ Surin Lee¹ Jae Yong Lee¹

Abstract

Deep learning methods based on backward stochastic differential equations (BSDEs) have emerged as competitive alternatives to physics-informed neural networks (PINNs) for solving high-dimensional partial differential equations (PDEs). By leveraging probabilistic representations, BSDE approaches can avoid the curse of dimensionality and often admit second-order-free training objectives that do not require explicit Hessian evaluations. It has recently been established that the commonly used Euler–Maruyama (EM) time discretization induces an intrinsic bias in BSDE training losses. While high-order schemes such as Heun can fully eliminate this bias, such schemes re-introduce second-order spatial derivatives and incur substantial computational overhead. In this work, we provide a principled analysis of EM-induced loss bias and propose an unbiased, second-order-free training framework that preserves the computational advantages of BSDE methods. Our code is available at <https://github.com/seojaemin22/Un-EM-BSDE>.

1. Introduction

The Deep Backward Stochastic Differential Equation (BSDE) method (Raissi, 2024; Han et al., 2018) serves as a critical complementary approach to Physics-Informed Neural Networks (PINNs) research (Raissi et al., 2019) by addressing the curse of dimensionality in high-dimensional PDEs. While standard PINN embeds the PDE residual in its loss function, the Deep BSDE method leverages the well-known mathematical connection between PDEs and stochastic differential equations (SDEs), reformulating the problem to enable efficient stochastic sampling along paths instead of relying on high-dimensional grid discretization,

¹Department of Artificial Intelligence, Chung-Ang University, Seoul, Republic of Korea. Correspondence to: Jae Yong Lee <jaeyong@cau.ac.kr>.

Table 1. Summary of bias properties, computational requirements, and training time.

Method	Unbiased	2nd-order-free	Time
EM-BSDE (Raissi, 2024)	✗	✓	1x
Shotgun (Xu & Zhang, 2025)	✗	✓	0.75x
Multi-Shot EM-BSDE	✗	✓	1.74x
Heun-BSDE (Park & Tu, 2025)	✓	✗	42.91x
FS-PINNs (Park & Tu, 2025)	✓	✗	32.07x
Un-EM-BSDE (ours)	✓	✓	1.79x

thereby offering a scalable and robust alternative for complex high-dimensional systems.

The BSDE methodology for solving high-dimensional PDEs approximates the solution by discretizing the paths of SDEs (Han et al., 2025). This process inherently requires enforcing temporal consistency through a self-consistency loss, where the solution values at two consecutive time steps are trained to satisfy the discretized BSDE (Nikolas & Richter, 2023). Typically, the BSDE is discretized in time using the Euler–Maruyama (EM) integration scheme. This time discretization yields the so-called EM-BSDE method, which is then solved by imposing a self-consistency loss. However, this class of solvers suffers from a significant discretization-induced bias issue, which severely degrades optimization performance. As a solution, Park & Tu (2025) proposed a new Heun-BSDE method, combining the Stratonovich SDE formulation with the Stochastic Heun integration scheme. The Heun-BSDE approach eliminates the discretization-induced bias inherent in the conventional EM-BSDE method, leading to a substantial improvement in the accuracy of BSDE-based solvers.

However, the performance of the Heun-BSDE method is limited by two main issues. Firstly, the Heun-BSDE method fails to demonstrate a clear enhancement compared to the FS-PINNs (Forward SDE-based Physics-Informed Neural Networks) method, which minimizes the standard PINN loss using sampled data along the identical forward SDE trajectory. The primary reason for this limited improvement is that the Heun-BSDE approach fundamentally still involves minimizing an estimate of the squared PDE residual. Secondly, Heun-BSDE is significantly more expensive than the baseline EM-BSDE and also incurs a higher training time compared to FS-PINNs. As reported in Table 1, Heun-BSDE requires $42.91\times$ the training time of

EM-BSDE, compared to $32.07\times$ for FS-PINNs. This increased computational cost stems from the need for multiple second-order derivative calculations. Specifically, while FS-PINNs require computing the Hessian only once per PDE residual calculation, the Heun-BSDE method requires two Hessian computations for a single step within the stochastic Heun scheme. Consequently, given the lack of substantial performance gains and the increased computational burden relative to FS-PINNs, the Heun-BSDE method presents limited practical advantage.

In this paper, we present a fundamental solution to the discretization-induced bias problem, avoiding the limited empirical gains and the heavy second-order-derivative overhead, by introducing a randomized unbiased loss estimator formed as the product of two independent one-step errors:

- We propose an **Unbiased EM-BSDE** method (**Un-EM-BSDE**), which enables training free of bias. While motivated by the general philosophy of combining complementary objectives in [Hu et al. \(2025\)](#), our method introduces a novel loss formulation specifically designed to eliminate bias induced by discretization in EM-BSDE training.
- We theoretically demonstrate that the proposed no-bias estimator shows no significant difference in terms of variance compared to the conventional biased EM-BSDE method. Furthermore, we suggest the potential to more efficiently control variance and enhance solver stability by leveraging a Shotgun-inspired training strategy ([Xu & Zhang, 2025](#)).
- We experimentally confirm that the proposed method achieves the accuracy level of the Heun-BSDE and FS-PINNs, even though its training time remains similar to the EM-BSDE, as it does not require direct computation of the second-order derivative term. Furthermore, we will demonstrate that any biased method can be converted into an unbiased one via the proposed debiasing mechanism, thereby yielding improved performance.

2. Related Work

Physics-Informed Neural Networks. PINNs have recently gained significant attention as a powerful, mesh-free methodology for solving high-dimensional PDEs ([Raissi et al., 2019](#); [Sirignano & Spiliopoulos, 2018](#); [Karniadakis et al., 2021](#); [Hu et al., 2024b](#)). PINNs parameterize the solution of the PDE using a neural network and directly minimize the residual of the PDE along with the boundary and initial condition residuals as a loss function through gradient descent. This approach has demonstrated impressive results across a wide range of problems ([Raissi et al., 2018](#); [Kissas et al., 2020](#)). However, PINNs often suffer from trainability

and accuracy difficulties for high-frequency or multiscale solutions ([Wang et al., 2022b](#); [Rathore et al., 2024](#); [Wang et al., 2022a](#)). Stable and accurate high-dimensional PINN solvers therefore remain an active research direction.

Deep BSDE method. On the other hand, methodologies based on BSDEs are presented as a complementary approach for solving high-dimensional PDEs ([Zhang et al., 2026](#)). These methods seek to overcome the curse of dimensionality by recasting the PDE as a forward-backward SDE and deriving a trajectory-based loss. Several works, including [Han et al. \(2017; 2018\)](#); [Huré et al. \(2020\)](#); [Beck et al. \(2021\)](#); [Germain et al. \(2022\)](#), and [Neufeld et al. \(2025\)](#), utilize separate networks to predict the solution and gradient of each time step. In contrast, [Zhang & Cai \(2022\)](#); [Raissi \(2024\)](#), [Xu & Zhang \(2025\)](#), and [Han et al. \(2026\)](#) employ a unified model, thereby achieving a better representation of the gradient continuity across steps and enabling the retrieval of a visual trajectory. This BSDE-based approach offers different training dynamics compared to the optimization issues faced by PINNs. Therefore, research aimed at understanding the performance differences between the two methodologies is a crucial research direction in the development of high-dimensional PDE solvers.

Randomized estimation for high-order derivatives. To improve the computational efficiency of PINNs in high-dimensional PDE problems, stochastic estimation methodologies ([Hutchinson, 1989](#); [Bekas et al., 2007](#)) that reduce the cost of high-order differentiation are being actively studied. Among these, a representative approach is to incorporate stochastic estimation within Automatic Differentiation to obtain low-cost approximations of higher-order derivatives ([Oktay et al., 2021](#); [Shi et al., 2024](#)), or to modify the network architecture and optimization scheme so that stochastic estimation becomes an integral component of the PINN training process ([Hu et al., 2024a;b; 2025](#)). Our method employs a different form of stochastic estimation to eliminate the bias.

3. Preliminary

3.1. Problem setup

We study learning-based approximation of the solution $u : \mathcal{T} \times \Omega \rightarrow \mathbb{R}$, where $\mathcal{T} = [0, T_{\text{end}}]$ and $\Omega \subset \mathbb{R}^d$, for non-linear terminal-value PDEs. We write the PDE in operator form

$$\mathcal{L}[u](t, x) = \phi(t, x, u(t, x), \nabla u(t, x)), \quad (1)$$

where

$$\begin{aligned} \mathcal{L}[u](t, x) := & \partial_t u(t, x) + \mu(t, x) \cdot \nabla u(t, x) \\ & + \frac{1}{2} \text{Tr}[\sigma^T(\nabla^2 u)\sigma](t, x), \end{aligned} \quad (2)$$

for $(t, x) \in \mathcal{T} \times \Omega$. We impose the terminal condition $u(T_{\text{end}}, x) = g(x)$ for $x \in \Omega$. We denote by ∇u and $\nabla^2 u$ the spatial gradient and Hessian, respectively. Here $\mu : \mathcal{T} \times \Omega \rightarrow \mathbb{R}^d$ and $\sigma : \mathcal{T} \times \Omega \rightarrow \mathbb{R}^{d \times d}$ represent the drift and diffusion coefficients, and $\phi : \mathcal{T} \times \Omega \times \mathbb{R} \times \mathbb{R}^d \rightarrow \mathbb{R}$ captures the nonlinear terms depending on $(u, \nabla u)$.

Our objective is to learn an accurate approximation of the PDE solution within a neural-network hypothesis class. Specifically, we parameterize $u_\theta : \mathcal{T} \times \Omega \rightarrow \mathbb{R}$ where u_θ is a neural network with parameters $\theta \in \Theta$, and seek θ^* such that u_{θ^*} satisfies the terminal condition and approximately solves the operator equation $\mathcal{L}[u] = \phi$ over $\mathcal{T} \times \Omega$. In many applications, however, the primary quantity of interest is the initial-time value $u(0, x_0)$ for a prescribed initial state x_0 . This setting is common in BSDE-based high-dimensional terminal-value solvers, which often target $u(0, x_0)$ rather than a uniform approximation of the full solution field. Accordingly, we focus on training objectives and solver designs that are tailored to accurately recovering this initial-time quantity. This perspective also clarifies why we do not adopt standard collocation-based PINN formulations: enforcing the PDE uniformly across the entire spatio-temporal domain typically requires dense sampling in (t, x) , which can be computationally demanding and often difficult to optimize, especially as the dimension d grows.

A natural alternative is to avoid global domain-wide enforcement and instead impose the governing equation along representative trajectories of an underlying stochastic process. Specifically, by Itô's formula, one can verify that the solution u of (1) is associated with the following forward-backward stochastic differential equation (FBSDE) system

$$\begin{cases} dX_t = \mu(t, X_t)dt + \sigma(t, X_t)dW_t, \\ dY_t = \phi(t, X_t, Y_t, Z_t)dt + Z_t^T \sigma(t, X_t)dW_t, \end{cases} \quad (3)$$

for $t \in \mathcal{T}$, where $X_0 = x_0$, $Y_{T_{\text{end}}} = g(X_{T_{\text{end}}})$, and $\{W_t : t \in \mathcal{T}\}$ is a d -dimensional standard Brownian motion. In particular, the correspondence is given by

$$Y_t = u(t, X_t), \quad Z_t = \nabla u(t, X_t). \quad (4)$$

In the continuous-time formulation, the training objective is expressed through quantities defined on $\mathcal{T}$ and typically involves expectations and time-integrated terms along the stochastic dynamics. In practice, these continuous-time objects are evaluated via numerical approximation, which motivates introducing a finite set of time points to approximate the underlying integrals. Throughout this work, we adopt a standard time-stepping approximation: we select an integer N and consider the grid $t_n := n\Delta t$ with $\Delta t := T_{\text{end}}/N$. This choice is made for clarity of exposition and computational convenience, and the same construction extends in a straightforward manner to non-uniform grids or alternative quadrature/time-sampling schemes.

Correspondingly, we work with Brownian increments $\Delta W_n := W_{t_{n+1}} - W_{t_n} \sim \mathcal{N}(0, \Delta t I_d)$ on the chosen grid and represent the dynamics through step-wise update maps. Specifically, we discretize the FBSDE using the Euler-Maruyama scheme and apply the corresponding one-step update

$$F_n(x) := x + \mu(t_n, x)\Delta t + \sigma(t_n, x)\Delta W_n, \quad (5)$$

$$B_n(x; u) := u(t_n, x) + \phi_u(t_n, x)\Delta t + \nabla u(t_n, x)^T \sigma(t_n, x)\Delta W_n, \quad (6)$$

where $\phi_u(t, x) := \phi(t, x, u(t, x), \nabla u(t, x))$. Using F_n and B_n , we introduce a one-step self-consistency error for the Euler-Maruyama discretization:

$$\text{err}_n^{\text{EM}}(x; u) := \frac{u(t_{n+1}, F_n(x)) - B_n(x; u)}{\Delta t}. \quad (7)$$

The corresponding one-step EM-BSDE loss is defined by:

$$\ell_{\text{EM}}(\theta; n, x) := \mathbb{E} [|\text{err}_n^{\text{EM}}(x; u_\theta)|^2], \quad (8)$$

where the expectation is taken with respect to ΔW_n . Aggregating along the forward trajectory $\hat{X}_{n+1} = F_n(\hat{X}_n)$ with $\hat{X}_0 = x_0$, we obtain the EM-BSDE objective:

$$L_{\text{EM}}(\theta) := \frac{1}{N} \sum_{n=0}^{N-1} \mathbb{E}_{\hat{X}_n} [\ell_{\text{EM}}(\theta; n, \hat{X}_n)]. \quad (9)$$

Based on this construction, [Raissi \(2024\)](#) approximate the solution of (3) by minimizing $L_{\text{EM}}(\theta)$ over the network parameters θ .

3.2. Bias of the BSDE loss

Recent results have shown that one-step self-consistency losses, when constructed from time-discretized BSDE updates, can exhibit a *discretization-induced bias* at finite step size Δt , in the sense that they are biased estimators of the corresponding continuous-time self-consistency objective implied by the PDE/FBSDE correspondence. In particular, the EM-BSDE loss is generally biased.

Lemma 3.1 (Bias of the EM-BSDE loss ([Park & Tu, 2025](#))). *Suppose that μ, σ are bounded and u_θ is $C^{1,2}$. Then we have:*

$$\begin{aligned} \ell_{\text{EM}}(\theta; n, x) &= ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 \\ &\quad + \frac{1}{2} \text{Tr}[(\sigma^T (\nabla^2 u_\theta) \sigma)^2](t_n, x) \\ &\quad + O(\Delta t^{1/2}). \end{aligned} \quad (10)$$

One way to address this issue is to modify the underlying stochastic calculus used in the formulation. Specifically, [Park & Tu \(2025\)](#) proposes rewriting the dynamics

in Stratonovich form and adopting a stochastic Heun integrator, which yields an alternative one-step update and the corresponding self-consistency error. The resulting Heun-BSDE loss is bias-free. See Appendix A.1 for the explicit definition of ℓ_{Heun} .

Lemma 3.2 (Unbiasedness of the Heun-BSDE loss (Park & Tu, 2025)). *Suppose that μ, σ , and ϕ_{u_θ} are all in $C^{1,1}$, and u_θ is in $C^{1,3}$. Then we have:*

$$\ell_{\text{Heun}}(\theta; n, x) = ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + O(\Delta t^{1/2}). \quad (11)$$

However, the Stratonovich-based Heun construction comes with a practical limitation. The Itô-to-Stratonovich conversion introduces correction terms that involve second-order spatial derivatives, so implementing the Heun-BSDE loss may require evaluating $\nabla^2 u_\theta$. This substantially increases the per-iteration cost and makes it comparable to PDE-residual-based methods.

Another way to address this issue is to keep the same one-step time-stepping scheme, but to reduce the discretization-induced bias by leveraging multiple independent Monte Carlo samples at each time step. In the Shotgun construction, we evaluate the one-step self-consistency error using M i.i.d. Brownian increments and aggregate them into a single averaged error (Xu & Zhang, 2025). To formalize this, for any random quantity ξ , we define the Shotgun averaging operator as the empirical average of M i.i.d. replicates:

$$\text{Shot}_M[\xi] := \frac{1}{M} \sum_{m=1}^M \xi_m, \quad (12)$$

where $\xi_1, \dots, \xi_M \stackrel{\text{i.i.d.}}{\sim} \xi$. Using this operator, we now define the one-step Shotgun loss by

$$\ell_{\text{SG}}^M(\theta; n, x) := \mathbb{E} \left[\left| \text{Shot}_M [\text{err}_n^{\text{SG}}(x; u_\theta)] \right|^2 \right], \quad (13)$$

where $\text{err}_n^{\text{SG}}(x; u_\theta)$ denotes the Brownian-increment-free one-step self-consistency error (see Appendix A.1 for the precise definition), and the expectation is taken with respect to the M i.i.d. Brownian increments used in $\text{Shot}_M[\cdot]$.

This multi-shot averaging leads to a reduced discretization-induced bias. We analyze the Shotgun construction of Xu & Zhang (2025) and prove the following bias expansion: compared to the standard EM-BSDE loss, the bias of ℓ_{SG}^M scales down by a factor of $1/M$.

Proposition 3.3 (Bias of the Shotgun loss). *Suppose that μ, σ are bounded and u_θ is $C^{1,2}$. Let $\tau > 0$ denote the time increment used in the Shotgun construction. Then we have:*

$$\begin{aligned} \ell_{\text{SG}}^M(\theta; n, x) &= ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 \\ &\quad + \frac{1}{2M} \text{Tr}[(\sigma^T (\nabla^2 u_\theta) \sigma)^2](t_n, x) \\ &\quad + O(\tau). \end{aligned} \quad (14)$$

Similarly, we can apply the same multi-shot averaging operator to the standard EM one-step error. This yields the Multi-Shot EM one-step loss.

$$\ell_{\text{SEM}}^M(\theta; n, x) := \mathbb{E} \left[\left| \text{Shot}_M [\text{err}_n^{\text{EM}}(x; u_\theta)] \right|^2 \right]. \quad (15)$$

This Multi-Shot EM loss exhibits the same bias reduction as the Shotgun loss. We show that this bias is reduced by a factor of $1/M$ in the following proposition.

Proposition 3.4 (Bias of the Multi-Shot EM loss). *Suppose that μ, σ are bounded and u_θ is $C^{1,2}$. Then we have:*

$$\begin{aligned} \ell_{\text{SEM}}^M(\theta; n, x) &= ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 \\ &\quad + \frac{1}{2M} \text{Tr}[(\sigma^T (\nabla^2 u_\theta) \sigma)^2](t_n, x) \\ &\quad + O(\Delta t^{1/2}). \end{aligned} \quad (16)$$

The proof of Proposition 3.3 and Proposition 3.4 are presented in Appendix A.2

Overall, existing approaches exhibit a clear trade-off between bias and efficiency. Heun-BSDE removes the discretization-induced bias at the one-step level, but the resulting objective is significantly more expensive to evaluate. The Shotgun and Multi-Shot constructions offer a cheaper mitigation by averaging over M independent samples, reducing the bias roughly at a $1/M$ rate, but they do not eliminate it. These limitations motivate an alternative debiasing strategy that preserves the efficiency of EM-type updates while eliminating the discretization-induced bias.

4. Proposed Method

In this section, we propose an Un-EM-BSDE loss that removes the discretization-induced bias while keeping the computational cost comparable to the standard EM-BSDE objective. We also show that the resulting estimator has a variance comparable to that of the original EM-BSDE loss.

4.1. Un-EM-BSDE

Motivated by a bias-variance decomposition perspective and inspired by related constructions in Hu et al. (2025), we propose the following Un-EM-BSDE objective:

$$\begin{aligned} \ell_{\text{UEM}}^{M_1, M_2}(\theta, n, x) &= \mathbb{E}[(\text{Shot}_{M_1} [\text{err}_n^{\text{EM}}(x; u_\theta)]) \\ &\quad \times (\text{Shot}_{M_2} [\text{err}_n^{\text{EM}}(x; u_\theta)])]. \end{aligned} \quad (17)$$

The key idea is to form the objective using independent draws of the one-step error, so that the expectation is factorized. This is closely related to the classical sample-splitting principle in statistics, where independent samples

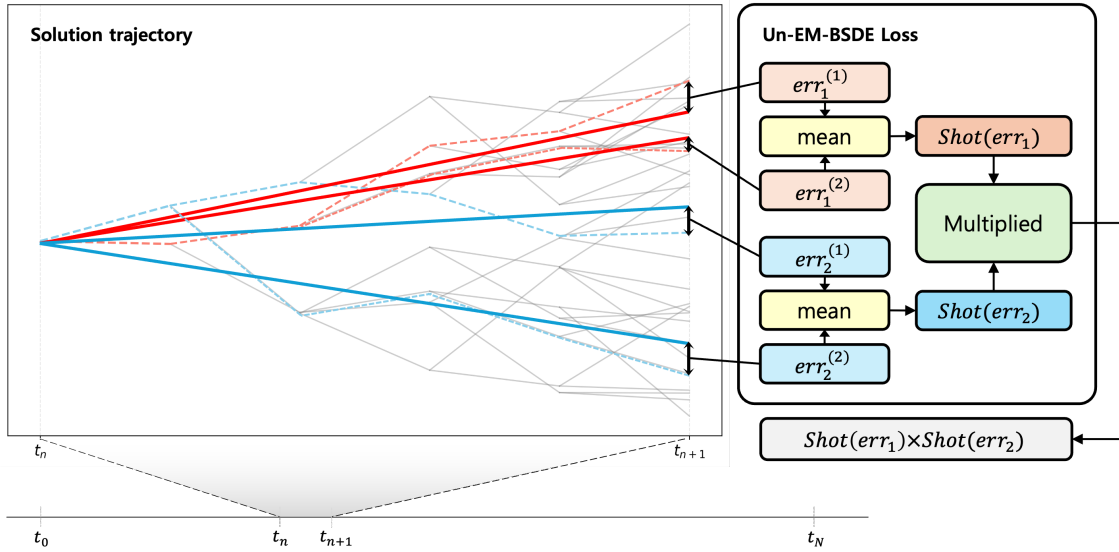


Figure 1. Illustration of the one-step loss computation for Un-EM-BSDE with $M_1 = M_2 = 2$. The many gray curves visualize the randomness of solution trajectories (possible SDE sample paths). The dashed curves denote a ground-truth trajectory, whereas the solid curves indicate the one-step predicted trajectory. The errors from the two independently formed groups are multiplied to produce an unbiased estimator.

are assigned to different factors so that expectations behave cleanly. In our setting, this principle is incorporated into the EM-BSDE objective in a problem-specific way to eliminate discretization-induced bias while preserving the second-order-free computational structure.

Lemma 4.1 (Unbiasedness of the Un-EM-BSDE loss). *Suppose that μ, σ are bounded and u_θ is $C^{1,2}$. Then we have*

$$\ell_{\text{UEM}}^{M_1, M_2}(\theta; n, x) = (\mathcal{L}[u_\theta](t_n, x) - \phi_{u_\theta}(t_n, x))^2 + O(\Delta t^{1/2}). \quad (18)$$

Having established the unbiasedness of the proposed construction, we further note that the resulting discrete-time objective is also consistent with its continuous-time counterpart in the limit $\Delta t \rightarrow 0$, analogously to the standard EM-BSDE and Heun-BSDE objectives.

Theorem 4.2 (Consistency of the Un-EM-BSDE loss). *Suppose that $f, g, \phi_{u_\theta} \in C^{1,0}$, $u_\theta \in C^{1,2}$, and $\Delta t \leq 1$. We have*

$$\begin{aligned} & \frac{1}{N} \sum_{n=0}^{N-1} \mathbb{E}_{\hat{X}_n} [\ell_{\text{UEM}}(\theta; n, \hat{X}_n)] \\ &= \frac{1}{|\mathcal{T}|} \int_{\mathcal{T}} \mathbb{E} \left[(\mathcal{L}[u_\theta] - \phi_{u_\theta})(t, X_t) \right]^2 dt + O(\Delta t^{1/2}). \end{aligned} \quad (19)$$

The proofs of Lemma 4.1 and Theorem 4.2 are provided in the appendix B.

4.2. Variance of the Un-EM-BSDE

While the independence-based construction removes the discretization-induced bias, debiasing via independent resampling can introduce a variance trade-off. In particular, replacing a biased second-moment objective by a cross-moment computed from independent draws typically involves products of noisy Monte Carlo quantities, which can amplify estimation variance unless the sampling budget is increased. Related bias-variance trade-offs for such cross-sample constructions have been discussed in Hu et al. (2025).

Motivated by this variance consideration, we now analyze the variance of the BSDE loss estimators. Let $\hat{\ell}_{\text{EM}}, \hat{\ell}_{\text{Heun}}, \hat{\ell}_{\text{SG}}^M, \hat{\ell}_{\text{SEM}}^M$, and $\hat{\ell}_{\text{UEM}}^{M_1, M_2}$ denote Monte Carlo estimators of $\ell_{\text{EM}}, \ell_{\text{Heun}}, \ell_{\text{SG}}^M, \ell_{\text{SEM}}^M$, and $\ell_{\text{UEM}}^{M_1, M_2}$, respectively. We summarize the resulting variance comparisons in the following theorem.

Theorem 4.3. *Define $\alpha = 2/M - 1/(2M_1) - 1/(2M_2)$ and $\beta = 1/(2M^2) - 1/(4M_1M_2)$. Assume $\beta > 0$ and $\alpha \geq 4/(3M + \beta M^4)$. Then, as $\tau \rightarrow 0, \Delta t \rightarrow 0$, the estimator variances satisfy*

$$\mathbb{V} [\hat{\ell}_{\text{UEM}}^{M_1, M_2}] \leq \mathbb{V} [\hat{\ell}_{\text{SG}}^M] = \mathbb{V} [\hat{\ell}_{\text{SEM}}^M] \leq \mathbb{V} [\hat{\ell}_{\text{EM}}]. \quad (20)$$

A particularly relevant setting is obtained by setting $M_1 = 1$ and $M_2 = 2$. In this case, the condition in Theorem 4.3 is satisfied with $M = 1$, implying that the resulting estimator variance is smaller than that of the standard EM-BSDE loss. Consequently, we can obtain an Un-EM-BSDE objective with improved estimator stability while incurring

Algorithm 1 Un-EM-BSDE loss algorithm

Input: batch size B , terminal time T_{end} , number of steps N , neural network $u_\theta(t, x)$, number of shots M_1, M_2
 Construct time grid: Define $\Delta t = T_{\text{end}}/N$, and define $T \in \mathbb{R}^{B \times (N+1)}$ by $T[b, n] = n\Delta t$ for each b, n
 Initialize candidate states: Allocate $X \in \mathbb{R}^{B \times (N+1) \times (M_1+M_2) \times d}$, and set $X[b, 0, 0] = x_0$ for each b
 Initialize main forward states: $(t, x) = (T[:, 0], X[:, 0, 0])$
for $n = 0$ **to** $N - 1$ **do**
 Sample i.i.d. Brownian increments: $\{\Delta W_{n,i}^{(b)}\}_{b \in [B], i \in [M_1+M_2]} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \Delta t I_d)$.
 Construct candidate next states: $X[:, n+1, i] = x + \mu(t, x)\Delta t + \sigma(t, x)\Delta W_{n,i}^{(b)}$ for each n, i
 Update main forward states: $(t, x) = (T[:, n+1], X[:, n+1, 0])$
end for
 Construct main forward trajectory: Define $X_{\text{main}} = X[:, :, 0]$
 Evaluate network outputs: Define $Y \in \mathbb{R}^{B \times (N+1) \times (M_1+M_2)}$ by $Y[b, n, i] = u_\theta(T[b, n], X[b, n, i])$ for each b, n, i
 Evaluate network gradients: Define $Z_{\text{main}} \in \mathbb{R}^{B \times (N+1) \times d}$ by $Z_{\text{main}}[b, n] = \nabla_x u_\theta(T, X_{\text{main}})[b, n]$ for each b, n
 Compute one-step BSDE prediction: Define $Y_{\text{main}} = Y[:, :, 0]$, and define $\hat{Y} \in \mathbb{R}^{B \times (N+1) \times (M_1+M_2)}$ by:

$$\hat{Y}[b, n+1, i] = Y_{\text{main}}[b, n] + \phi(T, X_{\text{main}}, Y_{\text{main}}, Z_{\text{main}})[b, n]\Delta t + [Z_{\text{main}}^T \sigma(T, X_{\text{main}})][b, n]\Delta W_{n,i}^{(b)} \quad \text{for each } b, n, i$$

Compute the per-sample step error and the group-wise averages: $\text{err}_{n,i}^{(b)} = Y[b, n+1, i] - \hat{Y}[b, n+1, i]$ for each b, n, i , and

$$\text{Shot_err}_{n,1}^{(b)} = \frac{1}{M_1} \sum_{i=0}^{M_1-1} \text{err}_{n,i}^{(b)}, \quad \text{Shot_err}_{n,2}^{(b)} = \frac{1}{M_2} \sum_{i=0}^{M_2-1} \text{err}_{n,M_1+i}^{(b)} \quad \text{for each } b$$

Compute BSDE self-consistency loss: $L_{\text{bsde}} = \frac{1}{B} \sum_{b=0}^{B-1} \sum_{n=0}^{N-1} \left(\text{Shot_err}_{n,1}^{(b)} \times \text{Shot_err}_{n,2}^{(b)} \right)$
if we use a soft constraint **then**
 Define $L_T = \frac{1}{B} \sum_{b=0}^{B-1} \left[(Y_N^{(b)} - g(T_N^{(b)}, X_N^{(b)}))^2 + \|Z_N^{(b)} - \nabla_x g(T_N^{(b)}, X_N^{(b)})\|_2^2 \right]$ where g is terminal function
 return L_{bsde}, L_T
else
 return L_{bsde}
end if

only a marginal additional sampling cost relative to the original EM-BSDE construction. The proof of Theorem 4.3 is provided in the Appendix C.

5. Experiment

In this section, we provide a comparative analysis of the proposed Un-EM-BSDE method against several approaches, including EM-BSDE, FS-PINNs, Shotgun, and the Heun-BSDE. Furthermore, we demonstrate that our debiasing mechanism is easily applicable to other biased methods, such as the Shotgun method. This highlights the applicability of our approach, showing its potential to be extended to a broad range of existing biased numerical frameworks.

We implement our proposed Un-EM-BSDE using Algorithm 1. Regarding the experimental setup, the hyperparameters are set to $M_1 = 5$ and $M_2 = 5$ based on empirically optimal performance results. Comprehensive details regarding the setup are provided in Appendix G.1. For baselines, EM-BSDE follows Algorithm 2, Heun-BSDE follows Algorithm 3, Shotgun follows Algorithm 4, and Multi-Shot EM-BSDE follows Algorithm 5. We use $M = 50$ for Shotgun, and $M = 10$ for Multi-Shot EM-BSDE to match the

inner-sampling budget of Un-EM-BSDE, i.e., $M_1 + M_2$. All derivatives in the experiments reported in this paper are computed using reverse-mode automatic differentiation.

5.1. Results on benchmark PDEs

We evaluate our method on a spectrum of problems—ranging from benchmark PDEs that fit the form of Eq. (1), including Hamilton–Jacobi–Bellman (HJB), Black–Scholes–Barenblatt (BSB), and Allen–Cahn (AC). By default, we compute the relative L_2 error (RL2) over the time grid along the trajectory. For AC, since a closed-form trajectory is not available, we instead report the relative error at the prescribed initial state. Full problem statements for HJB, BSB, and AC are provided in Appendix E.2.

The overall results are summarized in Table 2. Across most benchmark settings, our method achieves consistently strong accuracy and is typically among the top two methods. Figure 2 further shows a stable reduction in RL2 over time, indicating reliable convergence behavior. We consider both soft-constraint and hard-constraint formulations for enforcing terminal conditions, as both are widely used in practice and offer complementary trade-offs. While hard constraints enforce the conditions by construction, they can hinder op-

Table 2. Mean RL2 errors ($\times 10^{-2}$) with standard deviations ($\times 10^{-2}$) over three random seeds. The best and second-best results are highlighted in bold with underline and bold, respectively. For AC, the error is computed only at $t = 0$.

Cases	Biased methods			Unbiased methods		
	EM-BSDE	Shotgun	Multi-Shot EM-BSDE	Heun-BSDE	FS-PINNs	Un-EM-BSDE (Ours)
Soft constraint						
HJB	0.4055 $\pm$ 0.0107	1.1409 $\pm$ 0.1588	0.1617 $\pm$ 0.0144	0.1424 $\pm$ 0.0102	0.0867 $\pm$ 0.0018	0.1348 $\pm$ 0.0153
BSB	0.3483 $\pm$ 0.0008	39.9934 $\pm$ 23.6762	0.1046 $\pm$ 0.0024	0.1030 $\pm$ 0.0036	0.0478 $\pm$ 0.0015	0.0814 $\pm$ 0.0026
AC	0.0462 $\pm$ 0.0154	0.0951 $\pm$ 0.1270	0.0206 $\pm$ 0.0010	0.0774 $\pm$ 0.0027	0.0325 $\pm$ 0.0018	0.0147 $\pm$ 0.0044
BZ	5.3633 $\pm$ 0.0338	86.5295 $\pm$ 115.6677	5.3379 $\pm$ 0.0304	6.1697 $\pm$ 0.0845	174.4345 $\pm$ 120.3304	5.1778 $\pm$ 0.0533
PIDE	1.1094 $\pm$ 0.0753	1.2072 $\pm$ 0.0344	1.0771 $\pm$ 0.0314	0.9199 $\pm$ 0.0448	0.4973 $\pm$ 0.0097	1.0878 $\pm$ 0.0481
Hard constraint						
HJB	0.3433 $\pm$ 0.0051	0.2036 $\pm$ 0.0110	0.1172 $\pm$ 0.0086	0.2655 $\pm$ 0.0876	0.1876 $\pm$ 0.0487	0.1444 $\pm$ 0.0275
BSB	0.3456 $\pm$ 0.0002	0.1629 $\pm$ 0.0016	0.0739 $\pm$ 0.0002	0.0201 $\pm$ 0.0016	0.0048 $\pm$ 0.0009	0.0120 $\pm$ 0.0004
AC	0.0400 $\pm$ 0.0142	0.0136 $\pm$ 0.0016	0.0113 $\pm$ 0.0029	0.0912 $\pm$ 0.0004	0.0301 $\pm$ 0.0004	0.0034 $\pm$ 0.0024
BZ	1.1912 $\pm$ 0.0083	0.2383 $\pm$ 0.0403	1.1952 $\pm$ 0.0070	1.1885 $\pm$ 0.0027	0.2129 $\pm$ 0.0063	0.2279 $\pm$ 0.0010
PIDE	0.0374 $\pm$ 0.0155	0.4057 $\pm$ 0.0205	0.0245 $\pm$ 0.0087	0.1874 $\pm$ 0.0162	0.0137 $\pm$ 0.0022	0.0226 $\pm$ 0.0028

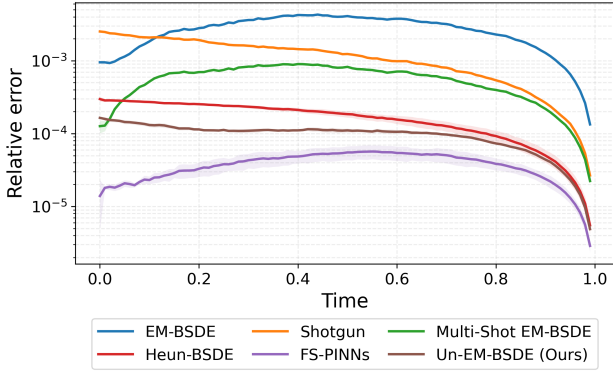


Figure 2. Time-resolved relative error on the BSB benchmark: relative error evaluated at each PDE time $t \in \mathcal{T}$ using the same models as in Table 2. The y-axis is plotted on a logarithmic scale.

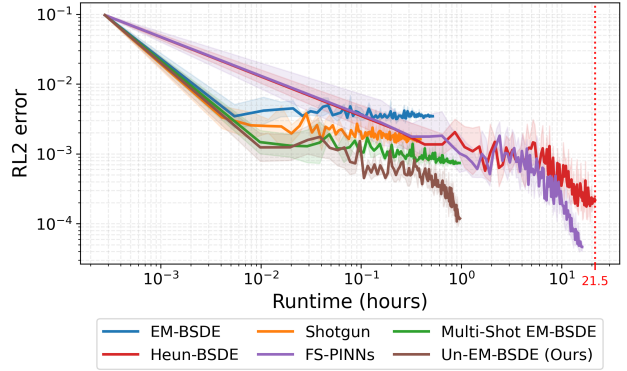


Figure 3. Runtime-accuracy on the BSB benchmark: RL2 recorded over runtime. Both axes are plotted on a log scale.

timization when an overly restrictive trial function unintentionally limits the admissible solution set (Liu et al., 2022). Soft constraints, in contrast, preserve flexibility but introduce the classical loss-balancing issue: the relative weighting between residual or self-consistency terms and constraint terms can strongly affect gradient magnitudes and training dynamics unless carefully tuned (Wang et al., 2022b; Bischof & Kraus, 2025). Furthermore, Figure 2 demonstrates that under the hard-constraint setting, our method remains stable and aligns well across the entire time domain.

Across our benchmarks, FS-PINNs train reliably under both formulations and often achieve the best accuracy, reflecting their strong empirical performance when optimization pro-

ceeds smoothly. For BSDE-based objectives, however, the choice of constraint enforcement has a more pronounced impact on training dynamics. This is particularly evident under soft constraints, where the balance between self-consistency and constraint penalties directly affects stability and convergence. In this respect, our method is comparatively robust: it performs consistently under both soft and hard constraints and reliably improves upon biased BSDE variants across problems. Under hard constraints, the removal of explicit constraint-loss terms simplifies the objective and reduces tuning overhead, making the benefit of randomized unbiased step-loss estimation more evident.

Additionally, our method demonstrates superior efficiency by avoiding explicit second-order derivative computations. In high dimensions, the presence of Hessian-related terms

can dominate the computational burden and may render other unbiased methods impractical to run at all. As shown in Figure 3, current unbiased methods, such as FS-PINNs and Heun-BSDE, require over an order of magnitude more runtime. By removing this bottleneck, our approach scales more favorably with dimensionality, providing a clear practical advantage when targeting high-dimensional PDEs. Moreover, as shown in Table 1, our method maintains a training cost comparable to standard EM-BSDE, while being substantially faster than other unbiased baselines. The resulting accuracy–runtime trade-off for the BSB hard-constraint setting illustrates that our method attains accuracy comparable to competing unbiased approaches at a noticeably lower computational cost.

5.2. Extensions to complex dynamics

We move beyond the benchmark PDEs and consider more practically motivated extensions where the standard theoretical assumptions and stability guarantees are less straightforward. Specifically, we study a problem from [Bender & Zhang \(2008\)](#) that, after the PDE-to-FBSDE transformation, yields a fully-coupled FBSDE because the diffusion coefficient σ depends on the unknown solution u ; and a Partial Integro-Differential Equations (PIDE) example from [Ye & Wang \(2024\)](#) whose jump-driven, discontinuous dynamics provide a stringent stress test for the proposed training objective. Formal definitions and full problem statements for the fully-coupled and PIDE extensions are provided in Appendix E.3 and Appendix E.4.

From an algorithmic standpoint, these extensions require only lightweight modifications. For the fully-coupled problem, we simply evaluate the network within the rollout since the coefficients depend on the current solution estimate. For the PIDE case, at each time step, we sample the number of jumps and their magnitudes and incorporate the resulting jump contribution in a scheme-agnostic manner. The last two rows for both the soft- and hard-constraint settings in Table 2 report the quantitative results for the BZ and PIDE cases. Overall, the soft-constraint formulation exhibits slightly less stable training dynamics in these more challenging settings, whereas the hard-constraint formulation remains stable and delivers competitive accuracy.

5.3. A generic debiasing wrapper for one-step losses

Although the debiasing mechanism was introduced in the context of the EM-BSDE loss, it applies more generally to any biased one-step training objective, provided that the one-step loss estimator can be evaluated under the same update rule with different time-step sizes. As an illustration, we apply this construction to the Shotgun objective, which averages M i.i.d. one-step errors but still carries discretization-induced bias at any fixed τ .

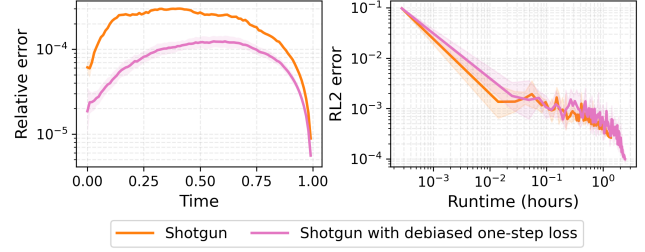


Figure 4. Relative error over PDE time (left) and RL2 over training runtime (right) for Shotgun and its debiased variant.

Concretely, applying the same randomized construction to Shotgun yields the following debiased one-step loss

$$\ell_{\text{USG}}^{M_1, M_2}(\theta, n, x) = \mathbb{E}[(\text{Shot}_{M_1}[\text{err}_n^{\text{SG}}(x; u_\theta)]) \times (\text{Shot}_{M_2}[\text{err}_n^{\text{SG}}(x; u_\theta)])]. \quad (21)$$

We then evaluate this plug-in variant on the BSB hard-constraint benchmark. Algorithmically, we retain the original Shotgun training pipeline and only replace its biased one-step objective with the debiased estimator above. Following the unbiased EM-BSDE implementation, we further adopt the same group-wise averaging and product aggregation used to form the final one-step loss, and set $M_1 = M_2 = 50$ throughout.

The results are visualized in Figure 4. Relative to the corresponding Shotgun run, the wrapper reduces the RL2 error by approximately $2.67\times$, while increasing the training time by $1.78\times$. This suggests that the wrapper can improve Shotgun’s accuracy with a moderate computational overhead.

These results suggest that the proposed construction is not tied to EM-BSDE: it can be plugged into other biased one-step losses and yields a clear accuracy gain at a moderate additional cost.

6. Conclusion

We presented an unbiased training objective for EM-BSDE-type solvers that removes discretization-induced bias without requiring explicit second-order derivative computations. Across standard benchmarks and more practically motivated extensions, the proposed method achieves consistently strong accuracy while keeping training time comparable to EM-BSDE and substantially lower than Hessian-based unbiased alternatives. Moreover, the same randomized construction can be used as a plug-in correction for other biased one-step objectives, as illustrated by debiasing the Shotgun loss. Future work includes integrating the proposed method with adaptive time-stepping to better allocate computation across time and improve efficiency on stiff or multi-scale dynamics. At the same time, our current theory is established under regularity and boundedness

assumptions, and therefore does not yet fully cover more challenging settings such as coupled FBSDEs and PIDEs. Extending the analysis to these broader BSDE-type problems remains an important direction for future work.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning for scientific computing, particularly in the numerical solution of high-dimensional partial differential equations. While such methods may contribute to improved modeling and simulation in areas such as finance and the physical sciences, the techniques developed here are general-purpose and do not raise specific ethical concerns beyond those commonly associated with advances in machine learning. The authors do not anticipate any immediate or unique negative societal impacts arising from this work.

Acknowledgments

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) [RS-2021-II211341, Artificial Intelligence Graduate School Program (Chung-Ang University)]. This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (RS-2025-02303239 and RS-2026-25497362).

References

- Beck, C., E. W., and Jentzen, A. Machine learning approximation algorithms for high-dimensional fully nonlinear partial differential equations and second-order backward stochastic differential equations. *J. Nonlinear Sci.*, 29(4):1563–1619, 2019. ISSN 0938-8974,1432-1467. doi: 10.1007/s00332-018-9525-3. URL <https://doi.org/10.1007/s00332-018-9525-3>.
- Beck, C., Becker, S., Cheridito, P., Jentzen, A., and Neufeld, A. Deep splitting method for parabolic PDEs. *SIAM J. Sci. Comput.*, 43(5):A3135–A3154, 2021. ISSN 1064-8275,1095-7197. doi: 10.1137/19M1297919. URL <https://doi.org/10.1137/19M1297919>.
- Bekas, C., Kokiopoulou, E., and Saad, Y. An estimator for the diagonal of a matrix. *Applied numerical mathematics*, 57(11-12):1214–1229, 2007.
- Bender, C. and Zhang, J. Time discretization and Markovian iteration for coupled FBSDEs. *Ann. Appl. Probab.*, 18(1):143–177, 2008. ISSN 1050-5164,2168-8737. doi: 10.1214/07-AAP448. URL <https://doi.org/10.1214/07-AAP448>.
- Bischof, R. and Kraus, M. A. Multi-objective loss balancing for physics-informed deep learning. *Comput. Methods Appl. Mech. Engrg.*, 439:Paper No. 117914, 28, 2025. ISSN 0045-7825,1879-2138. doi: 10.1016/j.cma.2025.117914. URL <https://doi.org/10.1016/j.cma.2025.117914>.
- Dangel, F., Siebert, T., Zeinhofer, M., and Walther, A. Collapsing taylor mode automatic differentiation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Germain, M., Pham, H., and Warin, X. Approximation Error Analysis of Some Deep Backward Schemes for Nonlinear PDEs. *SIAM Journal on Scientific Computing*, 44(1):A28–A56, 2022. doi: 10.1137/20M1355355.
- Han, J., Jentzen, A., et al. Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations. *Communications in mathematics and statistics*, 5(4):349–380, 2017.
- Han, J., Jentzen, A., and E. W. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, 2018. doi: 10.1073/pnas.1718942115.
- Han, J., Jentzen, A., et al. A brief review of the Deep BSDE method for solving high-dimensional partial differential equations. *arXiv preprint arXiv:2505.17032*, 2025.
- Han, J., Hu, W., Long, J., and Zhao, Y. Deep Picard Iteration for High-Dimensional Nonlinear PDEs. *SIAM Journal on Scientific Computing*, 48(1):C1–C24, 2026. doi: 10.1137/24M169312X. URL <https://doi.org/10.1137/24M169312X>.
- Hu, Z., Shi, Z., Karniadakis, G. E., and Kawaguchi, K. Hutchinson trace estimation for high-dimensional and high-order physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 424:116883, 2024a.
- Hu, Z., Shukla, K., Karniadakis, G. E., and Kawaguchi, K. Tackling the curse of dimensionality with physics-informed neural networks. *Neural Networks*, 176:106369, 2024b.
- Hu, Z., Yang, Z., Wang, Y., Karniadakis, G. E., and Kawaguchi, K. Bias-variance trade-off in physics-informed neural networks with randomized smoothing for high-dimensional PDEs. *SIAM J. Sci. Comput.*, 47(4):C846–C872, 2025. ISSN 1064-8275,1095-7197. doi: 10.1137/23M1621356. URL <https://doi.org/10.1137/23M1621356>.

- Huré, C., Pham, H., and Warin, X. Deep backward schemes for high-dimensional nonlinear PDEs. *Mathematics of Computation*, 89(324):1547–1579, 2020. doi: 10.1090/mcom/3514.
- Hutchinson, M. F. A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Communications in Statistics-Simulation and Computation*, 18(3):1059–1076, 1989.
- Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., and Yang, L. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021.
- Kissas, G., Yang, Y., Hwuang, E., Witschey, W. R., Detre, J. A., and Perdikaris, P. Machine learning in cardiovascular flows modeling: Predicting arterial blood pressure from non-invasive 4D flow MRI data using physics-informed neural networks. *Computer methods in applied mechanics and engineering*, 358:112623, 2020.
- Li, R., Ye, H., Jiang, D., Wen, X., Wang, C., Li, Z., Li, X., He, D., Chen, J., Ren, W., et al. A computational framework for neural network-based variational Monte Carlo with Forward Laplacian. *Nature Machine Intelligence*, 6(2):209–219, 2024.
- Liu, S., Hao, Z., Ying, C., Su, H., Zhu, J., and Cheng, Z. A Unified Hard-Constraint Framework for Solving Geometrically Complex PDEs. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=GNT5ntEGjD3>.
- Neufeld, A., Schmock, P., and Wu, S. Full error analysis of the random deep splitting method for nonlinear parabolic PDEs and PIDEs. *Communications in Nonlinear Science and Numerical Simulation*, 143:108556, 2025. doi: 10.1016/j.cnsns.2024.108556.
- Nikolas and Richter, L. R. Interpolating Between BSDEs and PINNs: Deep Learning for Elliptic and Parabolic Boundary Value Problems. *Journal of Machine Learning*, 2:31–64, 06 2023. doi: 10.4208/jml.220416.
- Oktay, D., McGreivy, N., Aduol, J., Beatson, A., and Adams, R. P. Randomized Automatic Differentiation. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=xpx9zj7CU1Y>.
- Park, S. and Tu, S. Integration Matters for Learning PDEs with Backwards SDEs. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=k4jglQCw0e>.
- Raissi, M. Forward-backward stochastic neural networks: deep learning of high-dimensional partial differential equations. In *Peter Carr Gedenkschrift: Research Advances in Mathematical Finance*, pp. 637–655. World Scientific, 2024.
- Raissi, M., Yazdani, A., and Karniadakis, G. E. Hidden fluid mechanics: A navier-stokes informed deep learning framework for assimilating flow visualization data. *arXiv preprint arXiv:1808.04327*, 2018.
- Raissi, M., Perdikaris, P., and Karniadakis, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.
- Rathore, P., Lei, W., Frangella, Z., Lu, L., and Udell, M. Challenges in Training PINNs: A Loss Landscape Perspective. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=mJGiFr8jLa>.
- Shi, Z., Hu, Z., Lin, M., and Kawaguchi, K. Stochastic taylor derivative estimator: Efficient amortization for arbitrary differential operators. *Advances in Neural Information Processing Systems*, 37:122316–122353, 2024.
- Sirignano, J. and Spiliopoulos, K. DGM: a deep learning algorithm for solving partial differential equations. *J. Comput. Phys.*, 375:1339–1364, 2018. ISSN 0021-9991,1090-2716. doi: 10.1016/j.jcp.2018.08.029. URL <https://doi.org/10.1016/j.jcp.2018.08.029>.
- Wang, C., Li, S., He, D., and Wang, L. Is L^2 Physics Informed Loss Always Suitable for Training Physics Informed Neural Network? *Advances in Neural Information Processing Systems*, 35:8278–8290, 2022a.
- Wang, S., Yu, X., and Perdikaris, P. When and why PINNs fail to train: a neural tangent kernel perspective. *J. Comput. Phys.*, 449:Paper No. 110768, 28, 2022b. ISSN 0021-9991,1090-2716. doi: 10.1016/j.jcp.2021.110768. URL <https://doi.org/10.1016/j.jcp.2021.110768>.
- Xu, W. and Zhang, W. A deep shotgun method for solving high-dimensional parabolic partial differential equations. *Journal of Scientific Computing*, 104(2):1–22, 2025.
- Ye, Z. and Wang, W. FBSJNN: A theoretically interpretable and efficiently deep learning method for solving partial integro-differential equations. *arXiv preprint arXiv:2412.11010*, 2024.
- Zhang, W. and Cai, W. FBSDE based neural network algorithms for high-dimensional quasilinear parabolic PDEs. *J. Comput. Phys.*, 470:Paper No. 111557, 14,

2022. ISSN 0021-9991,1090-2716. doi: 10.1016/j.jcp.2022.111557. URL <https://doi.org/10.1016/j.jcp.2022.111557>.

Zhang, W., Hu, Z., Cai, W., and Karniadakis, G. E. Deep Neural networks for solving high-dimensional parabolic partial differential equations. *arXiv preprint arXiv:2601.13256*, 2026.

Algorithm 2 EM-BSDE loss algorithm

Input: batch size B , terminal time T_{end} , number of steps N , neural network $u_\theta(t, x)$.
 Construct time grid: Define $\Delta t = T_{\text{end}}/N$, and define $T \in \mathbb{R}^{B \times (N+1)}$ by $T[b, n] = n\Delta t$ for each b, n .
 Initialize forward states: Allocate $X \in \mathbb{R}^{B \times (N+1) \times d}$, and set $X[b, 0] = x_0$ for each b, n .
for $n = 0$ **to** $N - 1$ **do**
 Sample i.i.d. Brownian increments: $\{\Delta W_n^{(b)}\}_{b \in [B]} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \Delta t I_d)$.
 Propagate forward states: $X[:, n+1] = X[:, n] + \mu(T[:, n], X[:, n])\Delta t + \sigma(T[:, n], X[:, n])\Delta W_n^{(b)}$
end for
 Evaluate network outputs: Define $Y \in \mathbb{R}^{B \times (N+1)}$ by $Y[b, n] = u_\theta(T[b, n], X[b, n])$ for each b, n .
 Evaluate network gradients: Allocate $Z \in \mathbb{R}^{B \times (N+1) \times d}$, and set $Z[b, n] = \nabla_x u_\theta(T[b, n], X[b, n])$ for each b, n .
 Compute one-step BSDE prediction: Define $\hat{Y} \in \mathbb{R}^{B \times (N+1)}$ by:

$$\hat{Y}[b, n+1] = Y[b, n] + \phi(T, X, Y, Z)[b, n]\Delta t + [Z^T \sigma(T, X)][b, n]\Delta W_n^{(b)} \quad \text{for each } b, n$$

Compute BSDE self-consistency loss: $L_{\text{bsde}} = \frac{1}{B} \sum_{b=0}^{B-1} \sum_{n=0}^{N-1} \left(Y[b, n+1] - \hat{Y}[b, n+1] \right)^2$
 Return the BSDE loss (with optional soft-constraint terminal condition loss) as in Algorithm 1

A. Background

This section provides additional background that is excluded from the main paper. Specifically, we give additional methodological details and provide algorithmic descriptions of how each method constructs its training loss, along with proofs for the bias of the resulting discretized objectives.

Throughout the paper, we present these algorithms in a batched form, which can be the most efficient option in principle for GPU training. For runtime reporting, however, we use a full-rollout implementation as the reference protocol, since it is the most commonly used choice in practice and batched execution can lead to substantial GPU-dependent differences in measured wall-clock time. Importantly, batching and full rollout evaluate the same discretized objective; they differ only in batching and parallelization of the same computations.

A.1. Deep BSDE methods

We can use Itô's formula to check why the solution of the FBSDE (3) comes from the PDE (1) as follows:

$$du(t, X_t) = (\partial_t u(t, X_t))dt + \nabla u(t, X_t) \cdot dX_t + \frac{1}{2}(dX_t)^T (\nabla^2 u(t, X_t))(dX_t) \quad (22)$$

$$= (\partial_t u(t, X_t))dt + [\nabla u \cdot (\mu dt + \sigma dW_t)](t, X_t) + \frac{1}{2}dW_t^T [\sigma^T (\nabla^2 u) \sigma](t, X_t)dW_t \quad (23)$$

$$= \left[\partial_t u + \mu \cdot \nabla u + \frac{1}{2} \text{Tr}[\sigma^T (\nabla^2 u) \sigma] \right] (t, X_t)dt + (\nabla u(t, X_t))^T \sigma(t, X_t)dW_t \quad (24)$$

$$= \mathcal{L}[u](t, X_t)dt + (\nabla u(t, X_t))^T \sigma(t, X_t)dW_t \quad (25)$$

$$= \phi(t, X_t, Y_t, Z_t)dt + Z_t^T \sigma(t, X_t)dW_t. \quad (26)$$

EM-BSDE (Raissi, 2024). We discretize the FBSDE using the Euler–Maruyama scheme and apply the corresponding one-step update.

$$\begin{cases} F_n(x) := x + \mu(t_n, x)\Delta t + \sigma(t_n, x)\Delta W_n, \\ B_n(x; u) := u(t_n, x) + \phi_u(t_n, x)\Delta t + \nabla u(t_n, x)^T \sigma(t_n, x)\Delta W_n. \end{cases} \quad (27)$$

Based on the resulting discrete FBSDE, we construct a one-step self-consistency error and the corresponding one-step loss.

$$\ell_{\text{EM}}(\theta; n, x) := \mathbb{E} [|\text{err}_n^{\text{EM}}(x; u_\theta)|^2], \quad \text{where } \text{err}_n^{\text{EM}}(x; u) := \frac{u(t_{n+1}, F_n(x)) - B_n(x; u)}{\Delta t}. \quad (28)$$

The overall training objective is formed by aggregating the per-step losses over the time grid (together with the terminal condition), leading to the implementation summarized in Algorithm 2.

Algorithm 3 Heun-BSDE loss algorithm

Input: batch size B , terminal time T_{end} , number of steps N , neural network $u_\theta(t, x)$.

Construct time grid and Initialize forward states as in Algorithm 2

Allocate intermediate states $\bar{X} \in \mathbb{R}^{B \times (N+1) \times d}$

for $n = 0$ **to** $N - 1$ **do**

Sample i.i.d. Brownian increments: $\{\Delta W_n^{(b)}\}_{b \in [B]} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \Delta t I_d)$.

Compute intermediate states: $\bar{X}[:, n+1] = X[:, n] + \Delta x$ where $\Delta x = \mu^{\text{Heun}}(T, X)[:, n] \Delta t + \sigma(T, X)[:, n] \Delta W_n^{(b)}$

Propagate forward states: $X[:, n+1] = X[:, n] + \frac{1}{2}(\Delta x + \bar{\Delta}x)$ where $\bar{\Delta}x = \mu^{\text{Heun}}(T, \bar{X})[:, n+1] \Delta t + \sigma(T, \bar{X})[:, n+1] \Delta W_n^{(b)}$

end for

Allocate $Y, \bar{Y} \in \mathbb{R}^{B \times (N+1)}$, $Z, \bar{Z} \in \mathbb{R}^{B \times (N+1) \times d}$, and $c, \bar{c} \in \mathbb{R}^{B \times (N+1)}$

Evaluate network outputs: $Y[b, n] = u_\theta(T, X)[b, n]$ and $\bar{Y}[b, n] = u_\theta(T, \bar{X})[b, n]$ for each b, n

Evaluate network gradients: $Z[b, n] = \nabla_x u_\theta(T, X)[b, n]$ and $\bar{Z}[b, n] = \nabla_x u_\theta(T, \bar{X})[b, n]$ for each b, n

Evaluate weighted network Laplacians: $c[b, n] = \text{Tr}[(\sigma^T (\nabla_x^2 u_\theta) \sigma)(T, X)[b, n]]$ and $\bar{c}[b, n] = \text{Tr}[(\sigma^T (\nabla_x^2 u_\theta) \sigma)(T, \bar{X})[b, n]]$

Compute one-step BSDE prediction: Define $\hat{Y} \in \mathbb{R}^{B \times (N+1)}$ by:

$$\begin{aligned} \hat{Y}[b, n+1] &= Y[b, n] + \Delta y & \text{where } \Delta y &= \phi_u^{\text{Heun}}(T, X)[b, n] \Delta t + [Z^T \sigma(T, X)][b, n] \Delta W_n^{(b)} \\ \hat{Y}[b, n+1] &= Y[b, n] + \frac{1}{2}(\Delta y + \bar{\Delta}y) & \text{where } \bar{\Delta}y &= \phi_u^{\text{Heun}}(T, \bar{X})[b, n+1] \Delta t + [\bar{Z}^T \sigma(T, \bar{X})][b, n+1] \Delta W_n^{(b)} \end{aligned}$$

Compute BSDE self-consistency loss: $L_{\text{bsde}} = \frac{1}{B} \sum_{b=0}^{B-1} \sum_{n=0}^{N-1} \left(Y[b, n+1] - \hat{Y}[b, n+1] \right)^2$

Return the BSDE loss (with optional soft-constraint terminal condition loss) as in Algorithm 1

Heun-BSDE (Park & Tu, 2025). While EM-BSDE relies on the Euler-Maruyama discretization of the forward SDE and the corresponding one-step BSDE update, the Heun-BSDE variant is based on a higher-order time integration that is consistent with a Stratonovich formulation. Specifically, we first rewrite the forward dynamics in Stratonovich form

$$\begin{cases} dX_t = \mu(t, X_t)dt + \sigma(t, X_t) \circ dW_t, \\ du(t, X_t) = [\phi_u - \frac{1}{2} \text{Tr}[\sigma^T (\nabla^2 u) \sigma]](t, X_t)dt + (\nabla u(t, X_t))^T \sigma(t, X_t) \circ dW_t. \end{cases} \quad (29)$$

We then discretize it using the stochastic Heun scheme and apply the corresponding one-step BSDE update

$$\begin{cases} \bar{F}_n(x) := x + \mu^{\text{Heun}}(t_n, x) \Delta t + \sigma(t_n, x) \Delta W_n, \\ \bar{B}_n(x; u) := u(t_n, x) + \phi_u^{\text{Heun}}(t_n, x) \Delta t + \nabla u(t_n, x)^T \sigma(t_n, x) \Delta W_n. \end{cases} \quad (30)$$

$$\begin{cases} F_n^{\text{Heun}}(x) := x + \frac{\mu^{\text{Heun}}(t_n, x) + \mu^{\text{Heun}}(t_{n+1}, \bar{F}_n(x))}{2} \Delta t + \frac{\sigma(t_n, x) + \sigma(t_{n+1}, \bar{F}_n(x))}{2} \Delta W_n, \\ B_n^{\text{Heun}}(x; u) := u(t_n, x) + \frac{\phi_u^{\text{Heun}}(t_n, x) + \phi_u^{\text{Heun}}(t_{n+1}, \bar{F}_n(x))}{2} \Delta t + \frac{[(\nabla u)^T \sigma](t_n, x) + [(\nabla u)^T \sigma](t_{n+1}, \bar{F}_n(x))}{2} \Delta W_n. \end{cases} \quad (31)$$

Here μ^{Heun} and ϕ_u^{Heun} incorporate the Stratonovich-Itô corrections associated with moving from Stratonovich dynamics to an Itô representation. These correction terms can be derived rigorously from the Stratonovich-Itô conversion formula via quadratic covariation. From a complementary, discrete-time viewpoint, the same structure is recovered in the Taylor-expansion analysis of the Heun-BSDE loss: the midpoint evaluation produces local contributions proportional to $\Delta W_n \Delta W_n^T$ (See (63) and (71)), whose expectation equals $I \Delta t$. For completeness, the explicit correction formulas are

$$\mu^{\text{Heun}} = \mu - \frac{1}{2} \sum_{i=1}^d (\partial_{x_i} \sigma) \sigma, \quad \phi_u^{\text{Heun}} = \phi_u - \frac{1}{2} \text{Tr}[\sigma^T (\nabla^2 u) \sigma] - \frac{1}{2} \sum_{i=1}^d (\nabla u)^T (\partial_{x_i} \sigma) \sigma. \quad (32)$$

Based on the resulting discrete FBSDE, we construct the one-step self-consistency error in the same manner as for EM-BSDE, and define the corresponding one-step loss:

$$\ell_{\text{Heun}}(\theta; n, x) := \mathbb{E} [|\text{err}_n^{\text{Heun}}(x; u_\theta)|^2], \quad \text{where } \text{err}_n^{\text{Heun}}(x; u) := \frac{u(t_{n+1}, F_n^{\text{Heun}}(x)) - B_n^{\text{Heun}}(x; u)}{\Delta t}. \quad (33)$$

Finally, we aggregate the per-step losses over the time grid and incorporate the terminal condition to form the overall training objective. The resulting Heun-BSDE objective is optimized using the procedure presented in Algorithm 3.

Algorithm 4 Shotgun loss algorithm

Input: batch size B , terminal time T_{end} , number of steps $N \geq 2$, neural network $u_\theta(t, x)$, finer time step τ , number of shots M .
 Construct time step : Define $\Delta t = T_{\text{end}}/(N - 1)$, and sample first time step $\{\Delta t_0^{(b)}\}_{b \in [B]} \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}(0, \Delta t)$
 Construct time grid : Define $T \in \mathbb{R}^{B \times (N+1)}$ by $T[b, 0] = 0$, $T[b, 1] = \Delta t_0$, $T[b, N] = T_{\text{end}}$, $T[b, n] = \Delta t_0 + (n - 1)\Delta t$ for each b, n , and define $\Delta t_n = T[:, n + 1] - T[:, n]$ for each n
 Initialize shotgun states : Allocate $X^+ \in \mathbb{R}^{B \times N \times M \times d}$ and $X^- \in \mathbb{R}^{B \times N \times M \times d}$
 Initialize main forward states : Allocate $X_{\text{main}} \in \mathbb{R}^{B \times (N+1) \times d}$, and set $X_{\text{main}}[b, 0] = x_0$ for each b, n
for $n = 0$ **to** $N - 1$ **do**
 Sample i.i.d. Brownian increments: $\{\Delta w_{n,i}^{(b)}\}_{b \in [B], i \in [M]} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \tau I_d)$ and $\{\Delta W_n^{(b)}\}_{b \in [B]} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \Delta t_n I_d)$.
 Compute shotgun states: $X^\pm[:, n, i] = X_{\text{main}}[:, n] + \mu(T, X_{\text{main}})[:, n]\tau \pm \sigma(T, X_{\text{main}})[:, n]\Delta w_{n,i}^{(b)}$ for each n, i
 Propagate main forward state: $X_{\text{main}}[:, n + 1] = X_{\text{main}}[:, n] + \mu(T, X_{\text{main}})[:, n]\Delta t_n + \sigma(T, X_{\text{main}})[:, n]\Delta W_n^{(b)}$
end for
 Allocate $Y^+ \in \mathbb{R}^{B \times N \times M}$, $Y^- \in \mathbb{R}^{B \times N \times M}$, $Y_{\text{main}} \in \mathbb{R}^{B \times (N+1)}$, and $Z_{\text{main}} \in \mathbb{R}^{B \times (N+1) \times d}$
 Evaluate network outputs : $Y^\pm[b, n, i] = u_\theta(T[b, n] + \tau, X^\pm[b, n, i])$ and $Y_{\text{main}}[b, n] = u_\theta(T, X_{\text{main}})[b, n]$ for each b, n, i
 Evaluate network gradients : $Z_{\text{main}}[b, n] = \nabla_x u_\theta(T, X_{\text{main}})[b, n]$ for each b, n
 Compute drift term : Define $\Phi \in \mathbb{R}^{B \times N \times M}$ by $\Phi[b, n] = \phi(T, X_{\text{main}}, Y_{\text{main}}, Z_{\text{main}})$ for each b, n
 Compute BSDE self-consistency loss

$$L_{\text{bsde}} = \frac{1}{B} \sum_{b=0}^{B-1} \sum_{n=0}^{N-1} \left(\frac{1}{M} \sum_{i=1}^M \left(\frac{Y^+ + Y^- - 2Y_{\text{main}}[b, n, i]}{2\tau} - \Phi[b, n] \right) \right)^2.$$

Return the BSDE loss (with optional soft-constraint terminal condition loss) as in Algorithm 1

Shotgun (Xu & Zhang, 2025). Compared with the EM-/Heun-BSDE constructions above, the Shotgun method introduces three key modifications. First, instead of using a fixed uniform grid shared across the batch, it randomizes the initial time step so that each sample in the batch is trained on a slightly shifted time grid. Second, on each such grid, it introduces a finer inner step size τ and constructs a Brownian-increment-free (i.e., ΔW -free) self-consistency error via antithetic pairing $(\Delta W, -\Delta W)$, so that the resulting error no longer depends explicitly on ΔW . Third, the per-step error is computed using multiple independent shots (i.e., multiple i.i.d. Brownian draws), and the resulting shot-wise errors are averaged to form the final loss, which reduces discretization-induced bias at a fixed time-stepping scheme.

Formally, let $\Delta t := T_{\text{end}}/(N - 1)$ and sample the first step as $t_0 = 0$, $t_1 \sim \mathcal{U}(0, \Delta t)$, $t_N = T_{\text{end}}$, and $t_n = t_1 + (n - 1)\Delta t$ for $n = 2, \dots, N - 1$. Denote $\Delta t_n := t_{n+1} - t_n$. We then sample independent Brownian increments $\Delta W_n \sim \mathcal{N}(0, \Delta t_n)$ for each n , and generate trajectories using the EM discretization:

$$\begin{cases} \hat{X}_{n+1} = \hat{X}_n + \mu(t_n, \hat{X}_n)\Delta t_n + \sigma(t_n, \hat{X}_n)\Delta W_n, \\ u(t_{n+1}, \hat{X}_{n+1}) = u(t_n, \hat{X}_n) + \phi_u(t_n, \hat{X}_n, u(t_n, \hat{X}_n), \nabla u(t_n, \hat{X}_n))\Delta t_n + (\nabla u(t_n, \hat{X}_n))^T \sigma(t_n, \hat{X}_n)\Delta W_n. \end{cases} \quad (34)$$

On each grid, we further introduce a finer time step τ and sample Brownian increment $\Delta w_n \sim \mathcal{N}(0, \tau I_d)$. Using the antithetic pair $(\Delta w_n, -\Delta w_n)$, we define $F_n^\pm(x) := x + \mu(t_n, x)\tau \pm \sigma(t_n, x)\Delta w_n$. We then construct a ΔW -free one-step self-consistency error and the corresponding one-step loss by averaging M independent shots:

$$\text{err}_n^{\text{SG}}(x; u) = \frac{u(t_n + \tau, F_n^+(x)) + u(t_n + \tau, F_n^-(x)) - 2u(t_n, x)}{2\tau} - \phi_u(t_n, x), \quad (35)$$

$$\ell_{\text{SG}}^M(\theta; n, x) := \mathbb{E} \left[\left| \text{Shot}_M [\text{err}_n^{\text{SG}}(x; u_\theta)] \right|^2 \right]. \quad (36)$$

The overall objective is obtained by aggregating the per-step losses along the time grid, together with the terminal condition; see Algorithm 4.

Multi-Shot EM-BSDE. The Multi-Shot EM-BSDE method can be viewed as the standard EM-BSDE baseline augmented only with the multiple independent shots mechanism from Shotgun. Equivalently, it matches the proposed Un-EM-BSDE construction with the debiasing step removed, retaining only the shot-averaging operator. Accordingly, the time discretization and the underlying one-step update are identical to those of EM-BSDE, and we continue to use the same EM one-step

Algorithm 5 Multi-Shot EM-BSDE loss algorithm

Input: batch size B , terminal time T_{end} , number of steps $N \geq 2$, neural network $u_\theta(t, x)$, finer time step τ , number of shots M .
 Construct time grid: Define $\Delta t = T_{\text{end}}/N$, and define $T \in \mathbb{R}^{B \times (N+1)}$ by $T[b, n] = n\Delta t$ for each b, n
 Initialize candidate states : Allocate $X \in \mathbb{R}^{B \times (N+1) \times M \times d}$, and set $X[b, 0, 0] = x_0$ for each b
 Initialize main forward states : $(t, x) = (T[:, 0], X[:, 0, 0])$
for $n = 0$ **to** $N - 1$ **do**
 Sample i.i.d. Brownian increments: $\{\Delta W_{n,i}^{(b)}\}_{b \in [B], i \in [M]} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \Delta t I_d)$.
 Construct candidate next states: $X[:, n+1, i] = x + \mu(t, x)\Delta t + \sigma(t, x)\Delta W_{n,i}^{(b)}$ for each n, i
 Update main forward states : $(t, x) = (T[:, n+1], X[:, n+1, 0])$
end for
 Construct main forward trajectory: Define $X_{\text{main}} = X[:, :, 0]$
 Evaluate network outputs : Define $Y \in \mathbb{R}^{B \times (N+1) \times M}$ by $Y[b, n, i] = u_\theta(T[b, n], X[b, n, i])$ for each b, n, i
 Evaluate network gradients : Define $Z_{\text{main}} \in \mathbb{R}^{B \times (N+1) \times d}$ by $Z_{\text{main}}[b, n] = \nabla_x u_\theta(T, X_{\text{main}})[b, n]$ for each b, n
 Compute one-step BSDE prediction : Define $Y_{\text{main}} = Y[:, :, 0]$, and define $\hat{Y} \in \mathbb{R}^{B \times (N+1) \times M}$ by:

$$\hat{Y}[b, n+1, i] = Y_{\text{main}}[b, n] + \phi(T, X_{\text{main}}, Y_{\text{main}}, Z_{\text{main}})[b, n]\Delta t + [Z_{\text{main}}^T \sigma(T, X_{\text{main}})][b, n]\Delta W_{n,i}^{(b)} \quad \text{for each } b, n, i$$

Compute BSDE self-consistency loss: $L_{\text{bsde}} = \frac{1}{B} \sum_{b=0}^{B-1} \sum_{n=0}^{N-1} \left(\frac{1}{M} \sum_{i=0}^{M-1} (Y[b, n+1, i] - \hat{Y}[b, n+1, i]) \right)^2$
 Return the BSDE loss (with optional soft-constraint terminal condition loss) as in Algorithm 1

self-consistency error err_n^{EM} . Following the same logic as in Shotgun, we evaluate this error using M independent shots and define the corresponding one-step loss via shot averaging:

$$\ell_{\text{SEM}}^M(\theta; n, x) := \mathbb{E} \left[\left| \text{Shot}_M [\text{err}_n^{\text{EM}}(x; u_\theta)] \right|^2 \right]. \quad (37)$$

We define the overall objective by integrating the per-step losses over the time grid, together with the terminal condition; see Algorithm 5.

A.2. Proof of bias for the existing BSDE loss

In this section, we restate and prove Lemma 3.1, Lemma 3.2, Proposition 3.3, and Proposition 3.4.

Lemma 3.1 (Bias of the EM-BSDE loss (Park & Tu, 2025)). *Suppose that μ, σ are bounded and u_θ is $C^{1,2}$. Then we have*

$$\ell_{\text{EM}}(\theta; n, x) = ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + \frac{1}{2} \text{Tr}[(\sigma^T (\nabla^2 u_\theta) \sigma)^2](t_n, x) + O(\Delta t^{1/2}). \quad (38)$$

Proof. By Taylor expansion, for any $n \in [N]$ and $x \in \mathbb{R}^d$, we obtain:

$$u_\theta(t_n + \Delta t, F_n(x)) - u_\theta(t_n, x) \quad (39)$$

$$\approx (\partial_t u_\theta(t_n, x))\Delta t + \nabla u_\theta(t_n, x) \cdot (F_n(x) - x) + \frac{1}{2} (F_n(x) - x)^T (\nabla^2 u_\theta(t_n, x)) (F_n(x) - x) \quad (40)$$

$$\approx (\partial_t u_\theta(t_n, x))\Delta t + [\nabla u_\theta \cdot (\mu \Delta t + \sigma \Delta W_n)](t_n, x) + \frac{1}{2} \Delta W_n^T [\sigma^T (\nabla^2 u_\theta) \sigma](t_n, x) \Delta W_n \quad (41)$$

$$\approx (\partial_t u_\theta(t_n, x))\Delta t + [\nabla u_\theta \cdot (\mu \Delta t + \sigma \Delta W_n)](t_n, x) + \frac{1}{2} \text{Tr}[\sigma^T (\nabla^2 u_\theta) \sigma](t_n, x) \Delta t \quad (42)$$

$$= \left[\partial_t u_\theta + \mu \cdot \nabla u_\theta + \frac{1}{2} \text{Tr}[\sigma^T (\nabla^2 u_\theta) \sigma] \right] (t_n, x) \Delta t + (\nabla u_\theta(t_n, x))^T \sigma(t_n, x) \Delta W_n \quad (43)$$

$$= \mathcal{L}[u_\theta](t_n, x) \Delta t + (\nabla u_\theta(t_n, x))^T \sigma(t_n, x) \Delta W_n. \quad (44)$$

Thus, we can write

$$u_\theta(t_n + \Delta t, F_n(x)) - u_\theta(t_n, x) = \mathcal{L}[u_\theta](t_n, x) \Delta t + (\nabla u_\theta(t_n, x))^T \sigma(t_n, x) \Delta W_n + \epsilon(n, x; u_\theta). \quad (45)$$

where

$$\epsilon(n, x; u_\theta) = \frac{1}{2} \Delta W_n^T [\sigma^T(\nabla^2 u_\theta) \sigma](t_n, x) \Delta W_n - \frac{1}{2} \text{Tr}[\sigma^T(\nabla^2 u_\theta) \sigma](t_n, x) \Delta t \quad [\cdot: (42)] \quad (46)$$

$$+ \frac{1}{2} [\mu^T(\nabla^2 u_\theta) \mu](t_n, x) \Delta t^2 - \Delta W_n^T [\sigma^T(\nabla^2 u_\theta) \mu](t_n, x) \Delta t \quad [\cdot: (41)] \quad (47)$$

$$+ \frac{1}{2} (\partial_{tt} u_\theta(t_n, x)) \Delta t^2 + (\text{third- and higher-order terms}) \quad [\cdot: (40)]. \quad (48)$$

Denote $H_{u_\theta} = \sigma^T(\nabla^2 u_\theta) \sigma$, and let $\xi_n = (\Delta t)^{-1/2} \Delta W_n$. We have

$$\text{err}_n^{\text{EM}}(x; u_\theta) = \frac{u_\theta(t_n + \Delta t, F_n(x)) - B_n(x; u_\theta)}{\Delta t} \quad (49)$$

$$= \frac{(\mathcal{L}[u_\theta](t_n, x) - \phi(t_n, x; u_\theta)) \Delta t + \epsilon(n, x; u_\theta)}{\Delta t} \quad (50)$$

$$= [\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + \frac{1}{2} [\xi_n^T H_{u_\theta} \xi_n - \text{Tr}[H_{u_\theta}]](t_n, x) + O(\Delta t^{1/2}). \quad (51)$$

Therefore we have

$$\ell_{\text{EM}}(\theta; n, x) = \mathbb{E} [|\text{err}_n^{\text{EM}}(x; u_\theta)|^2] \quad (52)$$

$$= \mathbb{E} \left[\left| [\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + \frac{1}{2} [\xi_n^T H_{u_\theta} \xi_n - \text{Tr}[H_{u_\theta}]](t_n, x) \right|^2 \right] + O(\Delta t^{1/2}) \quad (53)$$

$$= ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + \frac{1}{4} \mathbb{E} \left[|[\xi_n^T H_{u_\theta} \xi_n - \text{Tr}[H_{u_\theta}]](t_n, x)|^2 \right] + O(\Delta t^{1/2}) \quad (54)$$

$$= ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + \frac{1}{2} \text{Tr}[H_{u_\theta}^2](t_n, x) + O(\Delta t^{1/2}). \quad (55)$$

□

Remark A.1. The bound in Lemma 3.1 is not tight in the strict sense. Indeed, since the Brownian increment satisfies $\mathbb{E}[\Delta W_n] = 0$, the $\Delta t^{3/2}$ -order contributions appearing in the term in (47) and in the third-order remainder in (48) vanish after taking expectation once multiplied with the corresponding deterministic factors. Consequently, under an exact expectation, the remainder can be sharpened from $O(\Delta t^{1/2})$ to $O(\Delta t)$.

However, in practice, we do not have access to the exact conditional expectation when estimating the one-step loss, and the above cancellation cannot be exploited. As a result, ΔW_n should be viewed as a random variable of scale $\Delta t^{1/2}$ (i.e., $\Delta W_n = \Delta t^{1/2} \xi$ with $\xi \sim \mathcal{N}(0, I)$), and the corresponding $O(\Delta t^{1/2})$ control represents the effective bound relevant for stochastic estimation.

Lemma 3.2 (Unbiasedness of the Heun-BSDE loss (Park & Tu, 2025)). *Suppose that μ, σ , and ϕ_{u_θ} are all in $C^{1,1}$, and u_θ is in $C^{1,3}$. Then we have:*

$$\ell_{\text{Heun}}(\theta; n, x) = ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + O(\Delta t^{1/2}). \quad (56)$$

Proof. By Taylor expansion, for any $n \in [N]$ and $x \in \mathbb{R}^d$, we obtain

$$\mu^{\text{Heun}}(t_n + \Delta t, \bar{F}_n(x)) = \mu^{\text{Heun}}(t_n, x) + O(\Delta t^{1/2}), \quad (57)$$

$$\sigma(t_n + \Delta t, \bar{F}_n(x)) = \sigma(t_n, x) + \sum_{i=1}^d \partial_{x_i} \sigma(t_n, x) (\sigma(t_n, x) \Delta W_n)_i + O(\Delta t), \quad (58)$$

$$\phi_{u_\theta}^{\text{Heun}}(t_n + \Delta t, \bar{F}_n(x)) = \phi_{u_\theta}^{\text{Heun}}(t_n, x) + O(\Delta t^{1/2}), \quad (59)$$

$$[(\nabla u_\theta)^T \sigma](t_n + \Delta t, \bar{F}_n(x)) = [(\nabla u_\theta)^T \sigma](t_n, x) + \sum_{i=1}^d \partial_{x_i} [(\nabla u_\theta)^T \sigma](t_n, x) (\sigma(t_n, x) \Delta W_n)_i + O(\Delta t). \quad (60)$$

Denote $H_{u_\theta} = \sigma^T (\nabla^2 u_\theta) \sigma$ and let $\xi_n = (\Delta t)^{-1/2} \Delta W_n$. Then we have

$$F_n^{\text{Heun}}(x) - x \quad (61)$$

$$= \frac{\mu^{\text{Heun}}(t_n, x) + \mu^{\text{Heun}}(t_n + \Delta t, \bar{F}_n(x))}{2} \Delta t + \frac{\sigma(t_n, x) + \sigma(t_n + \Delta t, \bar{F}_n(x))}{2} \Delta W_n \quad (62)$$

$$= \left(\mu^{\text{Heun}}(t_n, x) + \frac{1}{2} \sum_{i=1}^d \partial_{x_i} \sigma(t_n, x) (\sigma(t_n, x) \xi_n)_i \xi_n \right) \Delta t + \sigma(t_n, x) \Delta W_n + O(\Delta t^{3/2}). \quad (63)$$

$$B_n^{\text{Heun}}(x; u_\theta) - u_\theta(t_n, x) \quad (64)$$

$$= \frac{\phi_{u_\theta}^{\text{Heun}}(t_n, x) + \phi_{u_\theta}^{\text{Heun}}(t_n + \Delta t, \bar{F}_n(x))}{2} \Delta t + \frac{[(\nabla u_\theta)^T \sigma](t_n, x) + [(\nabla u_\theta)^T \sigma](t_n + \Delta t, \bar{F}_n(x))}{2} \Delta W_n \quad (65)$$

$$= \left(\phi_{u_\theta}^{\text{Heun}}(t_n, x) + \frac{1}{2} \sum_{i=1}^d \partial_{x_i} [(\nabla u_\theta)^T \sigma](t_n, x) (\sigma(t_n, x) \xi_n)_i \xi_n \right) \Delta t \quad (66)$$

$$+ [(\nabla u_\theta)^T \sigma](t_n, x) \Delta W_n + O(\Delta t^{3/2}) \quad (67)$$

$$= \left(\phi_{u_\theta}^{\text{Heun}}(t_n, x) + \frac{1}{2} \sum_{i=1}^d [\partial_{x_i} (\nabla u_\theta(t_n, x))^T \sigma(t_n, x) + (\nabla u_\theta(t_n, x))^T \partial_{x_i} \sigma(t_n, x)] (\sigma(t_n, x) \xi_n)_i \xi_n \right) \Delta t \quad (68)$$

$$+ [(\nabla u_\theta)^T \sigma](t_n, x) \Delta W_n + O(\Delta t^{3/2}) \quad (69)$$

$$= \left[\phi_{u_\theta}^{\text{Heun}} + \frac{1}{2} \xi_n^T H_{u_\theta} \xi_n + \frac{1}{2} \sum_{i=1}^d (\nabla u_\theta)^T \partial_{x_i} \sigma (\sigma \xi_n)_i \xi_n \right] (t_n, x) \Delta t \quad (70)$$

$$+ [(\nabla u_\theta)^T \sigma](t_n, x) \Delta W_n + O(\Delta t^{3/2}). \quad (71)$$

And by Taylor expansion, we have

$$u_\theta(t_n + \Delta t, F_n^{\text{Heun}}(x)) - u_\theta(t_n, x) \quad (72)$$

$$= (\partial_t u_\theta(t_n, x)) \Delta t + \nabla u_\theta \cdot (F_n^{\text{Heun}}(x) - x) + \frac{1}{2} (F_n^{\text{Heun}}(x) - x)^T \nabla^2 u_\theta(t_n, x) (F_n^{\text{Heun}}(x) - x) + O(\Delta t^{3/2}) \quad (73)$$

$$= \left[\partial_t u_\theta + \mu^{\text{Heun}} \cdot \nabla u_\theta + \frac{1}{2} \xi_n^T H_{u_\theta} \xi_n + \frac{1}{2} \sum_{i=1}^d (\nabla u_\theta)^T \partial_{x_i} \sigma (\sigma \xi_n)_i \xi_n \right] (t_n, x) \Delta t \quad (74)$$

$$+ [(\nabla u_\theta)^T \sigma](t_n, x) \Delta W_n + O(\Delta t^{3/2}). \quad (75)$$

Therefore, we have

$$\text{err}_n^{\text{Heun}}(x; u_\theta) = \frac{u_\theta(t_n + \Delta t, F_n^{\text{Heun}}(x)) - B_n^{\text{Heun}}(x; u_\theta)}{\Delta t} \quad (76)$$

$$= \frac{(u_\theta(t_n + \Delta t, F_n^{\text{Heun}}(x)) - u_\theta(t_n, x)) - (B_n^{\text{Heun}}(x; u_\theta) - u_\theta(t_n, x))}{\Delta t} \quad (77)$$

$$= [\partial_t u_\theta + \mu^{\text{Heun}} \cdot \nabla u_\theta - \phi_{u_\theta}^{\text{Heun}}](t_n, x) + O(\Delta t^{1/2}) \quad (78)$$

$$= \left[\partial_t u_\theta + \mu \cdot \nabla u_\theta + \frac{1}{2} \text{Tr}[H_{u_\theta}] - \phi_{u_\theta} \right] (t_n, x) + O(\Delta t^{1/2}) \quad (79)$$

$$= [\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + O(\Delta t^{1/2}), \quad (80)$$

$$\therefore \ell_{\text{Heun}}(\theta; n, x) = \mathbb{E} \left[|\text{err}_n^{\text{Heun}}(x; u_\theta)|^2 \right] = ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + O(\Delta t^{1/2}). \quad (81)$$

□

Proposition 3.3 (Bias of the Shotgun loss (Xu & Zhang, 2025)). *Suppose that μ, σ are bounded and u_θ is $C^{1,2}$. Let $\tau > 0$ denote the time increment used in the Shotgun construction. Then we have:*

$$\ell_{\text{SG}}^M(\theta; n, x) = ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + \frac{1}{2M} \text{Tr}[(\sigma^T (\nabla^2 u_\theta) \sigma)^2](t_n, x) + O(\tau). \quad (82)$$

Proof. Define $x^\pm := x + \mu(t_n, x)\tau \pm \sigma(t_n, x)\Delta w_n$. Then, using the same derivation as in (45), we obtain

$$u_\theta(t_n + \tau, x^\pm) - u_\theta(t_n, x) = \mathcal{L}[u_\theta](t_n, x)\tau \pm (\nabla u_\theta(t_n, x))^T \sigma(t_n, x)\Delta w_n + \epsilon^\pm(n, x; u_\theta), \quad (83)$$

where

$$\epsilon^\pm(n, x; u_\theta) = \frac{1}{2}\Delta w_n^T [\sigma^T(\nabla^2 u_\theta)\sigma](t_n, x)\Delta w_n - \frac{1}{2}\text{Tr}[\sigma^T(\nabla^2 u_\theta)\sigma](t_n, x)\tau \quad (84)$$

$$+ \frac{1}{2}[\mu^T(\nabla^2 u_\theta)\mu](t_n, x)\tau^2 \mp \Delta w_n^T [\sigma^T(\nabla^2 u_\theta)\mu](t_n, x)\tau \quad (85)$$

$$+ \frac{1}{2}(\partial_{tt}u_\theta(t_n, x))\tau^2 + (\text{third- and higher-order terms}). \quad (86)$$

Denote $H_{u_\theta} = \sigma^T(\nabla^2 u_\theta)\sigma$, and let $\xi_n = \tau^{-1/2}\Delta w_n$. We have

$$\text{err}_n^{\text{SG}}(x; u_\theta) = \frac{u_\theta(t_n + \tau, x^+) + u_\theta(t_n + \tau, x^-) - 2u_\theta(t_n, x)}{2\tau} - \phi_{u_\theta}(t_n, x) \quad (87)$$

$$= [\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + \frac{\epsilon^+(n, x; u_\theta) + \epsilon^-(n, x; u_\theta)}{2\tau} \quad (88)$$

$$= [\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + \frac{1}{2}[\xi_n^T H_{u_\theta} \xi_n - \text{Tr}[H_{u_\theta}]](t_n, x) + O(\tau). \quad (89)$$

Given M independent samples $\{\Delta w_{n,i}\}_{i=1}^M \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \tau I_d)$, let $\xi_{n,i} = \tau^{-1/2}\Delta w_{n,i}$. Then the one-step loss is formed as follows:

$$\ell_{\text{SG}}^M(\theta; n, x) = \mathbb{E} \left[\left| \text{Shot}_M [\text{err}_n^{\text{SG}}(x; u_\theta)] \right|^2 \right] \quad (90)$$

$$= \mathbb{E} \left[\left| \frac{1}{M} \sum_{i=1}^M \left([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + \frac{1}{2}[\xi_{n,i}^T H_{u_\theta} \xi_{n,i} - \text{Tr}[H_{u_\theta}]](t_n, x) \right) \right|^2 \right] + O(\tau) \quad (91)$$

$$= \mathbb{E} \left[\left| [\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + \frac{1}{2M} \sum_{i=1}^M [\xi_{n,i}^T H_{u_\theta} \xi_{n,i} - \text{Tr}[H_{u_\theta}]](t_n, x) \right|^2 \right] + O(\tau) \quad (92)$$

$$= ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + \frac{1}{4M^2} \sum_{i=1}^M \mathbb{E} \left[|[\xi_{n,i}^T H_{u_\theta} \xi_{n,i} - \text{Tr}[H_{u_\theta}]](t_n, x)|^2 \right] + O(\tau) \quad (93)$$

$$= ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + \frac{1}{2M} \text{Tr}[H_{u_\theta}^2](t_n, x) + O(\tau). \quad (94)$$

□

Remark A.2. The $O(\tau)$ remainder in Proposition 3.3 reflects a stronger cancellation mechanism than the one discussed around (47)–(48). There, the $\Delta t^{3/2}$ -order contributions are proportional to the Brownian increment and disappear only after taking expectation, so the cancellation is not directly available at the level of a single Monte Carlo realization. In contrast, the Shotgun approximation in (88) combines the two branch errors through $\epsilon^+ + \epsilon^-$, where $\epsilon^\pm$ correspond to the antithetic pair $(\Delta w_n, -\Delta w_n)$ at step size τ . This pairing enforces a pathwise sign cancellation: all terms that are odd in Δw_n cancel in $\epsilon^+ + \epsilon^-$, which removes the leading $\tau^{3/2}$ -order contributions directly at the discrete level. Consequently, the effective remainder improves to $O(\tau)$ without relying on access to the exact expectation.

Proposition 3.4 (Bias of the Multi-Shot EM loss). *Suppose that μ, σ are bounded and u_θ is $C^{1,2}$. Then we have:*

$$\ell_{\text{SEM}}^M(\theta; n, x) = ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + \frac{1}{2M} \text{Tr}[(\sigma^T(\nabla^2 u_\theta)\sigma)^2](t_n, x) + O(\Delta t^{1/2}). \quad (95)$$

Proof. Since the Multi-Shot EM construction is obtained by applying $\text{Shot}_M[\cdot]$ to the standard EM one-step error, the bias expansion is a direct consequence of the EM error approximation in (51), combined with the shot-averaging argument used

for Shotgun. Given M i.i.d. samples $\{\Delta w_{n,i}\}_{i=1}^M \sim \mathcal{N}(0, \Delta t I_d)$, denote $H_{u_\theta} = \sigma^T(\nabla^2 u_\theta)\sigma$, and let $\xi_{n,i} = \Delta t^{-1/2} \Delta w_{n,i}$. Then we have:

$$\ell_{\text{SEM}}^M(\theta; n, x) = \mathbb{E} \left[\left| \text{Shot}_M [\text{err}_n^{\text{EM}}(x; u_\theta)] \right|^2 \right] \quad (96)$$

$$= \mathbb{E} \left[\left| \frac{1}{M} \sum_{i=1}^M \left([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + \frac{1}{2} [\xi_{n,i}^T H_{u_\theta} \xi_{n,i} - \text{Tr}[H_{u_\theta}]](t_n, x) \right) \right|^2 \right] + O(\Delta t^{1/2}) \quad (97)$$

$$= ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + \frac{1}{2M} \text{Tr}[H_{u_\theta}^2](t_n, x) + O(\Delta t^{1/2}). \quad (98)$$

□

B. Proofs of Lemma 4.1 and Theorem 4.2

In this section, we restate and prove Lemma 4.1 and Theorem 4.2.

Lemma 4.1 (Unbiasedness of the Un-EM-BSDE loss). *Suppose that μ, σ are bounded and u_θ is $C^{1,2}$. Then we have*

$$\ell_{\text{UEM}}^{M_1, M_2}(\theta; n, x) = ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + O(\Delta t^{1/2}). \quad (99)$$

Proof. Recall the EM one-step error $\text{err}_n^{\text{EM}}(x; u_\theta)$ characterized in (51). Given $M_1 + M_2$ independent samples $\{\Delta w_{n,i}\}_{i=1}^{M_1+M_2} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \Delta t I_d)$, let $\xi_{n,i} = \Delta t^{-1/2} \Delta w_{n,i}$. We apply the shot-averaging operator to $\text{err}_n^{\text{EM}}(x; u_\theta)$ and examine its expectation:

$$\mathbb{E} [\text{Shot}_{M_1} [\text{err}_n^{\text{EM}}(x; u_\theta)]] \quad (100)$$

$$= \frac{1}{M_1} \sum_{i=1}^{M_1} \mathbb{E} \left[[\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + \frac{1}{2} [\xi_{n,i}^T H_{u_\theta} \xi_{n,i} - \text{Tr}[H_{u_\theta}]](t_n, x) \right] + O(\Delta t^{1/2}) \quad (101)$$

$$= [\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + O(\Delta t^{1/2}), \quad (102)$$

$$\mathbb{E} [\text{Shot}_{M_2} [\text{err}_n^{\text{EM}}(x; u_\theta)]] \quad (103)$$

$$= \frac{1}{M_2} \sum_{i=M_1+1}^{M_1+M_2} \mathbb{E} \left[[\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + \frac{1}{2} [\xi_{n,i}^T H_{u_\theta} \xi_{n,i} - \text{Tr}[H_{u_\theta}]](t_n, x) \right] + O(\Delta t^{1/2}) \quad (104)$$

$$= [\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + O(\Delta t^{1/2}). \quad (105)$$

Using these two independent shot-averaged estimates, we can write the Un-EM-BSDE one-step loss as:

$$\ell_{\text{UEM}}^{M_1, M_2}(\theta, n, x) = \mathbb{E} [(\text{Shot}_{M_1} [\text{err}_n^{\text{EM}}(x; u_\theta)])(\text{Shot}_{M_2} [\text{err}_n^{\text{EM}}(x; u_\theta)])] \quad (106)$$

$$= \mathbb{E} [\text{Shot}_{M_1} [\text{err}_n^{\text{EM}}(x; u_\theta)]] \mathbb{E} [\text{Shot}_{M_2} [\text{err}_n^{\text{EM}}(x; u_\theta)]] \quad (107)$$

$$= ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + O(\Delta t^{1/2}). \quad (108)$$

□

Theorem 4.2 (Consistency of the Un-EM-BSDE loss). *Suppose that $\mu, \sigma, \phi_{u_\theta} \in C^{1,0}$, $u_\theta \in C^{1,2}$, and $\Delta t \leq 1$. We have*

$$\frac{1}{N} \sum_{n=0}^{N-1} \mathbb{E}_{\hat{X}_n} [\ell_{\text{UEM}}(\theta; n, \hat{X}_n)] = \frac{1}{|\mathcal{T}|} \int_{\mathcal{T}} \mathbb{E} [([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t, X_t))^2] dt + O(\Delta t^{1/2}). \quad (109)$$

Proof. The argument is identical to that of [Park & Tu \(2025, Theorem 4.2\)](#), except that Lemma 3.1 is substituted by Lemma 4.1. Concretely, by Lemma 4.1, we obtain

$$\frac{1}{N} \sum_{n=0}^{N-1} \mathbb{E}_{\hat{X}_n} [\ell_{\text{UEM}}(\theta; n, \hat{X}_n)] = \frac{1}{N} \sum_{n=0}^{N-1} \mathbb{E}_{\hat{X}_n} \left[([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, \hat{X}_n))^2 \right] + O(\Delta t^{1/2}). \quad (110)$$

Under the regularity assumptions $f, g, \phi_{u_\theta} \in C^{0,1}$ and $u_\theta \in C^{2,1}$, the map $(t, x) \mapsto ([\mathcal{L} - \phi_{u_\theta}](t, x))^2$ is Lipschitz continuous on $\mathcal{T} \times \mathbb{R}^d$. Moreover, the forward process $(X_t)_{t \in \mathcal{T}}$ is driven by the same Itô SDE as in [Park & Tu \(2025, Theorem 4.2\)](#). Therefore, the remainder of the proof in [Park & Tu \(2025, Theorem 4.2\)](#) carries over unchanged, and the claim follows. $\square$

C. Proof of Theorem 4.3

Theorem 4.3 . Define $\alpha = 2/M - 1/(2M_1) - 1/(2M_2)$ and $\beta = 1/(2M^2) - 1/(4M_1M_2)$. Assume $\beta > 0$ and $\alpha \geq 4/(3M + \beta M^4)$. Then, as $\Delta t \rightarrow 0$ and $\tau \rightarrow 0$, the estimator variances satisfy

$$\mathbb{V} [\hat{\ell}_{\text{UEM}}^{M_1, M_2}] \leq \mathbb{V} [\hat{\ell}_{\text{SG}}^M] = \mathbb{V} [\hat{\ell}_{\text{SEM}}^M] \leq \mathbb{V} [\hat{\ell}_{\text{EM}}]. \quad (111)$$

Proof. To establish the variance comparison, we begin by deriving explicit variance expansions for the four estimators under consideration. We use the same notation and conventions as in the preceding bias analyses and introduce no additional notation.

$$\mathbb{V} [\hat{\ell}_{\text{EM}}(\theta, n, x)] \quad (112)$$

$$= \mathbb{E} \left[\left([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + \frac{1}{2} [\xi_n^T H_{u_\theta} \xi_n - \text{Tr}[H_{u_\theta}]](t_n, x) \right)^4 \right] \quad (113)$$

$$- \left(([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + \frac{1}{2} \text{Tr}[H_{u_\theta}^2](t_n, x) \right)^2 + O(\Delta t^{1/2}) \quad (114)$$

$$= ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^4 + \frac{3}{2} ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 \mathbb{E} [([\xi_n^T H_{u_\theta} \xi_n - \text{Tr}[H_{u_\theta}]](t_n, x))^2] \quad (115)$$

$$+ \frac{1}{2} [\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) \mathbb{E} [([\xi_n^T H_{u_\theta} \xi_n - \text{Tr}[H_{u_\theta}]](t_n, x))^3] + \frac{1}{16} \mathbb{E} [([\xi_n^T H_{u_\theta} \xi_n - \text{Tr}[H_{u_\theta}]](t_n, x))^4] \quad (116)$$

$$- ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^4 - ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 \text{Tr}[H_{u_\theta}^2](t_n, x) - \frac{1}{4} (\text{Tr}[H_{u_\theta}^2](t_n, x))^2 + O(\Delta t^{1/2}) \quad (117)$$

$$= 2([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 \text{Tr}[H_{u_\theta}^2](t_n, x) + 4[\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) \text{Tr}[H_{u_\theta}^3](t_n, x) + 3\text{Tr}[H_{u_\theta}^4](t_n, x) \quad (118)$$

$$+ \frac{1}{2} (\text{Tr}[H_{u_\theta}^2](t_n, x))^2 + O(\Delta t^{1/2}), \quad (119)$$

$$\mathbb{V} [\hat{\ell}_{\text{SG}}^M(\theta, n, x)] \quad (120)$$

$$= \mathbb{E} \left[\left([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + \frac{1}{2M} \sum_{i=1}^M [\xi_{n,i}^T H_{u_\theta} \xi_{n,i} - \text{Tr}[H_{u_\theta}]](t_n, x) \right)^4 \right] \quad (121)$$

$$- \left([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x)^2 + \frac{1}{2M} \text{Tr}[H_{u_\theta}^2](t_n, x) \right)^2 + O(\tau) \quad (122)$$

$$= ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^4 + \frac{3}{2M^2} ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 \sum_{i=1}^M \mathbb{E} [([\xi_{n,i}^T H_{u_\theta} \xi_{n,i} - \text{Tr}[H_{u_\theta}]](t_n, x))^2] \quad (123)$$

$$+ \frac{1}{2M^3} [\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) \sum_{i=1}^M \mathbb{E} [([\xi_{n,i}^T H_{u_\theta} \xi_{n,i} - \text{Tr}[H_{u_\theta}]](t_n, x))^3] \quad (124)$$

$$+ \frac{1}{16M^4} \sum_{i=1}^M \mathbb{E} [([\xi_{n,i}^T H_{u_\theta} \xi_{n,i} - \text{Tr}[H_{u_\theta}]](t_n, x))^4] \quad (125)$$

$$+ \frac{3}{8M^4} \sum_{1 \leq i < j \leq M} \mathbb{E} [([\xi_{n,i}^T H_{u_\theta} \xi_{n,i} - \text{Tr}[H_{u_\theta}]]^2 (\xi_{n,j}^T H_{u_\theta} \xi_{n,j} - \text{Tr}[H_{u_\theta}])^2](t_n, x)] \quad (126)$$

$$- ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^4 - \frac{1}{M} ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 \text{Tr}[H_{u_\theta}^2](t_n, x) - \frac{1}{4M^2} (\text{Tr}[H_{u_\theta}^2](t_n, x))^2 + O(\tau) \quad (127)$$

$$= \frac{2}{M} ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 \text{Tr}[H_{u_\theta}^2](t_n, x) + \frac{4}{M^2} [\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) \text{Tr}[H_{u_\theta}^3](t_n, x) + \frac{3}{M^3} \text{Tr}[H_{u_\theta}^4](t_n, x) \quad (128)$$

$$+ \frac{1}{2M^2} (\text{Tr}[H_{u_\theta}^2](t_n, x))^2 + O(\tau), \quad (129)$$

$$(130)$$

$$\mathbb{V} [\hat{\ell}_{\text{SEM}}^M(\theta, n, x)] \quad (131)$$

$$= \mathbb{E} \left[\left([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + \frac{1}{2M} \sum_{i=1}^M [\xi_{n,i}^T H_{u_\theta} \xi_{n,i} - \text{Tr}[H_{u_\theta}]](t_n, x) \right)^4 \right] \quad (132)$$

$$- \left(([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + \frac{1}{2M} \text{Tr}[H_{u_\theta}^2](t_n, x) \right)^2 + O(\Delta t^{1/2}) \quad (133)$$

$$= \frac{2}{M} ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 \text{Tr}[H_{u_\theta}^2](t_n, x) + \frac{4}{M^2} [\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) \text{Tr}[H_{u_\theta}^3](t_n, x) + \frac{3}{M^3} \text{Tr}[H_{u_\theta}^4](t_n, x) \quad (134)$$

$$+ \frac{1}{2M^2} (\text{Tr}[H_{u_\theta}^2](t_n, x))^2 + O(\Delta t^{1/2}), \quad (135)$$

$$(136)$$

$$\mathbb{V} [\hat{\ell}_{\text{UEM}}^{M_1, M_2}(\theta, n, x)] \quad (137)$$

$$= \mathbb{E} \left[\left([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + \frac{1}{2M_1} \sum_{i=1}^{M_1} [\xi_{n,i}^T H_{u_\theta} \xi_{n,i} - \text{Tr}[H_{u_\theta}]](t_n, x) \right)^2 \right] \quad (138)$$

$$\left([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x) + \frac{1}{2M_2} \sum_{i=M_1+1}^{M_1+M_2} [\xi_{n,i}^T H_{u_\theta} \xi_{n,i} - \text{Tr}[H_{u_\theta}]](t_n, x) \right)^2 \quad (139)$$

$$- ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^4 + O(\Delta t^{1/2}) \quad (140)$$

$$= \left(([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + \frac{1}{2M_1} \text{Tr}[H_{u_\theta}^2](t_n, x) \right) \left(([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 + \frac{1}{2M_2} \text{Tr}[H_{u_\theta}^2](t_n, x) \right) \quad (141)$$

$$- ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^4 + O(\Delta t^{1/2}) \quad (142)$$

$$= \left(\frac{1}{2M_1} + \frac{1}{2M_2} \right) ([\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x))^2 \text{Tr}[H_{u_\theta}^2](t_n, x) + \frac{1}{4M_1 M_2} (\text{Tr}[H_{u_\theta}^2](t_n, x))^2 + O(\Delta t^{1/2}). \quad (143)$$

Comparing the variance expansions above, we first observe that, as $\Delta t \rightarrow 0$ and $\tau \rightarrow 0$,

$$\mathbb{V}[\hat{\ell}_{\text{SG}}^M] = \mathbb{V}[\hat{\ell}_{\text{SEM}}^M] \quad (144)$$

Therefore, it remains only to prove

$$\mathbb{V}[\hat{\ell}_{\text{SEM}}^M] \leq \mathbb{V}[\hat{\ell}_{\text{EM}}] \quad \text{and} \quad \mathbb{V}[\hat{\ell}_{\text{UEM}}^{M_1, M_2}] \leq \mathbb{V}[\hat{\ell}_{\text{SG}}^M]. \quad (145)$$

Let $y := [\mathcal{L}[u_\theta] - \phi_{u_\theta}](t_n, x)$, $H := H_{u_\theta}(t_n, x)$, and $T_k := \text{Tr}[H^k]$. Under the variance expansions derived above, the two desired inequalities are equivalent to the nonnegativity of the following two quadratic polynomials in y :

$$Q_1(y) := 2 \left(1 - \frac{1}{M}\right) y^2 T_2 + 4 \left(1 - \frac{1}{M^2}\right) y T_3 + 3 \left(1 - \frac{1}{M^3}\right) T_4 + \frac{1}{2} \left(1 - \frac{1}{M^2}\right) T_2^2 \geq 0, \quad (146)$$

$$Q_2(y) := \alpha y^2 T_2 + \frac{4}{M^2} y T_3 + \frac{3}{M^3} T_4 + \beta T_2^2 \geq 0, \quad (147)$$

Here, $Q_1(y) \geq 0$ corresponds to $\mathbb{V}[\hat{\ell}_{\text{SEM}}^M] \leq \mathbb{V}[\hat{\ell}_{\text{EM}}]$, whereas $Q_2(y) \geq 0$ corresponds to $\mathbb{V}[\hat{\ell}_{\text{UEM}}^{M_1, M_2}] \leq \mathbb{V}[\hat{\ell}_{\text{SG}}^M]$.

For Q_1 , the case $M = 1$ gives $Q_1 \equiv 0$. For $M > 1$, the leading coefficient is nonnegative, and it suffices to show that its discriminant is nonpositive. The discriminant is

$$D_1 = 16 \left(1 - \frac{1}{M^2}\right)^2 T_3^2 - 8 \left(1 - \frac{1}{M}\right) T_2 \left[3 \left(1 - \frac{1}{M^3}\right) T_4 + \frac{1}{2} \left(1 - \frac{1}{M^2}\right) T_2^2 \right] \quad (148)$$

For Q_2 , since $\beta > 0$ and $\alpha \geq 4/(3M + \beta M^4) > 0$, its leading coefficient is nonnegative. Its discriminant is

$$D_2 = \left(\frac{4}{M^2} T_3 \right)^2 - 4\alpha T_2 \left(\frac{3}{M^3} T_4 + \beta T_2^2 \right). \quad (149)$$

Since H is symmetric, let $(\lambda_1, \dots, \lambda_d)$ be its real eigenvalues. Then

$$T_2^2 = \left(\sum_{i=1}^d \lambda_i^2 \right)^2 = \sum_{i=1}^d \lambda_i^4 + 2 \sum_{1 \leq i < j \leq d} \lambda_i^2 \lambda_j^2 \geq \sum_{i=1}^d \lambda_i^4 = T_4, \quad (150)$$

$$T_3^2 = \left(\sum_{i=1}^d (\lambda_i)(\lambda_i)^2 \right)^2 \leq \left(\sum_{i=1}^d \lambda_i^2 \right) \left(\sum_{i=1}^d (\lambda_i^2)^2 \right) = T_2 T_4 \quad [\because \text{Cauchy-Schwarz}]. \quad (151)$$

Using these inequalities and $T_2 \geq 0$, we obtain

$$D_1 \leq 16 \left(1 - \frac{1}{M^2}\right)^2 T_2 T_4 - 24 \left(1 - \frac{1}{M}\right) \left(1 - \frac{1}{M^3}\right) T_2 T_4 \quad (152)$$

$$= -8 \left(1 - \frac{1}{M}\right) \left(1 + \frac{1}{M} + \frac{5}{M^2} + \frac{2}{M^3}\right) T_2 T_4 \leq 0. \quad (153)$$

$$D_2 \leq \frac{16}{M^4} T_2 T_4 - 4\alpha \left(\frac{3}{M^3} + \beta \right) T_2 T_4 \leq 0 \quad (154)$$

where the last inequality follows from $\alpha \geq 4/(3M + \beta M^4) = (4/M^4)/(3/M^3 + \beta)$. Therefore $Q_1(y) \geq 0$ and $Q_2(y) \geq 0$ for all y , which proves $\mathbb{V}[\hat{\ell}_{\text{SEM}}^M] \leq \mathbb{V}[\hat{\ell}_{\text{EM}}]$ and $\mathbb{V}[\hat{\ell}_{\text{UEM}}^{M_1, M_2}] \leq \mathbb{V}[\hat{\ell}_{\text{SG}}^M]$. This concludes the proof. $\square$

D. Computation of $\mathbb{E}[(\xi^T H \xi - \text{Tr}[H])^n]$

Suppose $\xi \sim \mathcal{N}(0, I_d)$ and $H \in \mathbb{R}^{d \times d}$ is a symmetric matrix. Then, by the spectral theorem, we can write $H = U^T \Lambda U$ where U is an orthogonal matrix and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$. If we define $Z := U\xi$, then $Z \sim \mathcal{N}(0, I_d)$, because of rotation invariance. Hence we obtain

$$\xi^T H \xi = (U\xi)^T \Lambda (U\xi) = \sum_{i=1}^d \lambda_i Z_i^2. \quad (155)$$

Thus, we can denote

$$X := \xi^T H \xi - \text{Tr}[H] = \sum_{i=1}^d \lambda_i (Z_i^2 - 1). \quad (156)$$

Note that the moment generating function of X is $M_X(t) := \mathbb{E}[e^{tX}]$, and the cumulant generating function is

$$K_X(t) := \log M_X(t) = \sum_{r \geq 1} \frac{\kappa_r t^r}{r!}. \quad (157)$$

where the coefficients κ_r are the cumulants. We can compute $\mathbb{E}[X^n]$ using the following:

$$\mathbb{E}[X^n] = \left. \frac{d^n}{dt^n} M_X(t) \right|_{t=0} = \left. \frac{d^n}{dt^n} e^{K_X(t)} \right|_{t=0} \quad (158)$$

$$= e^{K_X(0)} \sum_{\pi \in \Pi_n} \prod_{B \in \pi} K_X^{(|B|)}(0) \quad [\cdot : \text{Faà di Bruno's formula}] \quad (159)$$

$$= \sum_{\pi \in \Pi_n} \prod_{B \in \pi} \kappa_{|B|}, \quad (160)$$

where Π_n denotes the set of all partitions of the set $\{1, \dots, n\}$.

Since $Z_i^2 \sim \chi_1^2$ so that $\log \mathbb{E}[e^{tZ_i^2}] = -\frac{1}{2} \log(1 - 2t)$, we obtain

$$K_X(t) = \log \mathbb{E} \left[\exp \left(t \sum_{i=1}^d \lambda_i (Z_i^2 - 1) \right) \right] \quad (161)$$

$$= \log \prod_{i=1}^d e^{-t\lambda_i} \mathbb{E} \left[e^{t\lambda_i Z_i^2} \right] \quad [\cdot : \text{independence}] \quad (162)$$

$$= \sum_{i=1}^d \left(-t\lambda_i - \frac{1}{2} \log(1 - 2t\lambda_i) \right). \quad (163)$$

Since $\log(1 - 2t\lambda_i) = -\sum_{r \geq 1} \frac{(2t\lambda_i)^r}{r}$ for $|2t\lambda_i| < 1$, we obtain

$$K_X(t) = \sum_{i=1}^d \sum_{r \geq 2} \frac{(2t\lambda_i)^r}{2r} = \sum_{r \geq 2} \frac{2^{r-1}}{r} t^r \sum_{i=1}^d \lambda_i^r. \quad (164)$$

Matching coefficients with (157) yields

$$\kappa_1 = 0, \quad \kappa_r = 2^{r-1}(r-1)! \text{Tr}[H^r] \quad \text{for } r \geq 2. \quad (165)$$

Therefore we can compute $\mathbb{E}[X^n]$ using (160), such as

$$\mathbb{E}[X] = \kappa_1 = 0. \quad (166)$$

$$\mathbb{E}[X^2] = \kappa_1^2 + \kappa_2 = 2\text{Tr}[H^2]. \quad (167)$$

$$\mathbb{E}[X^3] = \kappa_1^3 + 3\kappa_1\kappa_2 + \kappa_3 = 8\text{Tr}[H^3]. \quad (168)$$

$$\mathbb{E}[X^4] = \kappa_1^4 + 6\kappa_1^2\kappa_2 + 3\kappa_2^2 + 4\kappa_1\kappa_3 + \kappa_4 = 48\text{Tr}[H^4] + 12(\text{Tr}[H^2])^2. \quad (169)$$

E. Experimental Details

We employ deep neural networks to approximate the solution for all benchmark problems. Following the experimental setup in [Ye & Wang \(2024\)](#), the model for the partial integro-differential equation consists of two hidden layers with 256 neurons each and utilizes the Leaky-ReLU activation function. It is trained for 10,000 iterations using the Adam optimizer with a piecewise-constant scheduler. For the remaining benchmarks, we adopt a deeper configuration. These models comprise four hidden layers with 512 neurons each and the Mish activation function. Training is conducted for 100,000 iterations using the Adam optimizer with a cosine decay scheduler and an initial learning rate of 10^{-3} . We conduct experiments under two different settings for terminal conditions, namely hard and soft constraints. For the stochastic simulations, we use a trajectory batch size of 64, translating to 64 realizations of the underlying Brownian motions. All simulations are implemented using the JAX library with 32-bit floating-point precision (float32) and performed on an NVIDIA RTX 6000 Ada Generation GPU, except for the experiment in Section 5.3, which uses 64-bit floating-point precision (float64) on an NVIDIA A100 GPU due to the increased numerical sensitivity of replacing Shotgun’s one-step loss with the debiased loss.

We fix $N = 100$ for all experiments to maintain consistent input dimensions, except for the Shotgun method. For Shotgun, we strictly follow the original literature by setting $\tau = 4^{-5}$ and $N = 10$. Despite the shorter trajectory, the inherent δ parameter provides sufficient regularization to compensate for the reduced discretization, thereby preserving both gradient flow and accuracy.

E.1. Relative L_2 error metric

To evaluate performance, test trajectories are generated using the exact solution when available; otherwise, they are constructed via a Brownian Bridge to ensure high-fidelity forward trajectories. The reported accuracy is the average relative L_2 error (RL2) calculated across 256 test trajectories, defined as:

$$\text{RL2} = \sqrt{\frac{\sum_{n=0}^N |u(t_n, X_n) - u_\theta(t_n, X_n)|^2}{\sum_{n=0}^N |u(t_n, X_n)|^2}} \quad (170)$$

E.2. Benchmark equations

In this section, we focus on benchmark PDEs within the general class given in (1).

Hamilton–Jacobi–Bellman Equation (HJB). We consider the d -dimensional quadratic HJB equation. For $t \in [0, T_{\text{end}}]$ and $x \in \mathbb{R}^d$, the PDE is defined as:

$$\partial_t u(t, x) + \text{Tr}[\nabla^2 u(t, x)] = \|\nabla u(t, x)\|^2. \quad (171)$$

with the terminal condition $u(T_{\text{end}}, x) = g(x)$. By (3), the above PDE admits the following FBSDE formulation:

$$dX_t = \sqrt{2}dW_t. \quad (172)$$

$$dY_t = \|Z_t\|^2 dt + \sqrt{2}Z_t^T dW_t. \quad (173)$$

where $X_0 = \xi \in \mathbb{R}^d$ and $Y_{T_{\text{end}}} = g(X_{T_{\text{end}}})$. For our experiments, we set $d = 100$, $T_{\text{end}} = 1$, $\xi = (0, 0, \dots, 0) \in \mathbb{R}^{100}$, and $g(x) = \ln(0.5(1 + \|x\|^2))$. The exact solution to this equation is given by

$$u(t, x) = -\ln \left(\mathbb{E} \left[\exp \left(-g(x + \sqrt{2}W_{T_{\text{end}}-t}) \right) \right] \right). \quad (174)$$

Black–Scholes–Barenblatt Equation (BSB). We consider the d -dimensional BSB equation:

$$\partial_t u + \frac{1}{2} \text{Tr}[\alpha^2 \text{diag}(x^2) \nabla^2 u(t, x)] = r(u(t, x) - \nabla u(t, x)^T x). \quad (175)$$

where $t \in [0, T_{\text{end}}]$ and $x \in \mathbb{R}^d$, with the terminal condition $u(T_{\text{end}}, x) = g(x)$. Applying (3) yields the following FBSDE formulation:

$$dX_t = \alpha \text{diag}(X_t) dW_t \quad (176)$$

$$dY_t = r(Y_t - Z_t^T X_t) dt + \alpha Z_t^T \text{diag}(X_t) dW_t, \quad (177)$$

where $X_0 = \xi \in \mathbb{R}^d$ and $Y_{T_{\text{end}}} = g(X_{T_{\text{end}}})$. For our experiments, we set $d = 100$, $T_{\text{end}} = 1$, $\xi = (1, 0.5, 1, 0.5, \dots, 1, 0.5) \in \mathbb{R}^d$, $\alpha = 0.4$, $r = 0.05$, and $g(x) = \|x\|^2$. The exact solution to this equation is given by

$$u(t, x) = \exp((r + \alpha^2)(T_{\text{end}} - t)) g(x). \quad (178)$$

Allen–Cahn Equation (AC). We consider the d -dimensional AC equation, which is a semilinear parabolic PDE. For $t \in [0, T_{\text{end}}]$ and $x \in \mathbb{R}^d$, the equation is defined as:

$$\frac{\partial u}{\partial t} + \frac{1}{2} \Delta u = u^3 - u, \quad (179)$$

with the terminal condition $u(T_{\text{end}}, x) = g(x)$. From (3), we have:

$$dX_t = dW_t, \quad (180)$$

$$dY_t = (Y_t^3 - Y_t)dt + Z_t^T dW_t, \quad (181)$$

where $X_0 = \xi \in \mathbb{R}^d$ and $Y_{T_{\text{end}}} = g(X_{T_{\text{end}}})$. For our experiments, we set $d = 20$, $T_{\text{end}} = 0.3$, $\xi = (0, 0, \dots, 0) \in \mathbb{R}^{20}$, and $g(x) = (2 + 0.4\|x\|^2)^{-1}$.

Due to the absence of an exact solution for the AC equation, the relative error in Table 2 was calculated at $t = 0$ by comparing the model's output with a reference value of 0.30879. This benchmark value was obtained via the Branching Diffusion Method (Beck et al., 2019), which is considered an unbiased approach for this problem.

E.3. Fully-Coupled FBSDE (BZ)

We consider the following non-linear PDE:

$$\partial_t u(t, x) + \frac{1}{2} \text{Tr}[\sigma(t, x, u)^T \nabla^2 u(t, x) \sigma(t, x, u)] = \phi(t, x, u), \quad (182)$$

with the terminal condition $u(T_{\text{end}}, x) = g(x)$, where the respective terms are given by:

$$\sigma(t, x, u) = \alpha u(t, x) I_d, \quad (183)$$

$$\phi(t, x, u) = ru - \frac{1}{2} e^{-3r(T_{\text{end}}-t)} \alpha^2 \left(D \sum_{j=1}^d \sin(x_j) \right)^3. \quad (184)$$

The PDE can be reformulated as the following fully-coupled FBSDE:

$$dX_t = \alpha Y_t dW_t, \quad (185)$$

$$dY_t = \left[rY_t - \frac{1}{2} e^{-3r(T_{\text{end}}-t)} \alpha^2 \left(D \sum_{j=1}^d \sin(X_{j,t}) \right)^3 \right] dt + Z_t^T \sigma(t, X_t, Y_t) dW_t, \quad (186)$$

where $X_0 = \xi \in \mathbb{R}^d$ and $Y_{T_{\text{end}}} = g(X_{T_{\text{end}}})$. For our experiments, we set $d = 100$, $T_{\text{end}} = 1$, $r = 0.1$, $\alpha = 0.3$, $D = 0.1$, $\xi = (\pi/2, \pi/2, \dots, \pi/2)$, and $g(x) = D \sum_{j=1}^d \sin(x_j)$. The above FBSDE has the exact solution:

$$u(t, x) = e^{-r(T_{\text{end}}-t)} D \sum_{j=1}^d \sin(x_j). \quad (187)$$

The BZ is introduced to evaluate the algorithm's performance on a fully-coupled system, where the forward and backward dynamics are interdependent. This formulation is adapted from Bender & Zhang (2008) and has been utilized as a benchmark for high-dimensional BSDE solvers, as mentioned in Park & Tu (2025).

E.4. Partial integro-differential equation (PIDE)

We consider a high-dimensional PIDE that incorporates jump components. For $t \in [0, T_{\text{end}}]$ and $x \in \mathbb{R}^d$, the governing equation is defined as follows:

$$\begin{cases} \frac{\partial u}{\partial t}(t, x) + \frac{1}{2} \text{Tr}(\tau^2 \nabla_x^2 u(t, x)) + \left\langle \frac{1}{2} \epsilon x, \nabla_x u(t, x) \right\rangle \\ + \int_{\mathbb{R}} (u(t, x + e) - u(t, x) - \langle e, \nabla_x u(t, x) \rangle) \nu(de) = \lambda(\mu_\phi^2 + \sigma_\phi^2) + \tau^2 + \frac{\epsilon}{d} \|x\|^2, \\ u(T_{\text{end}}, x) = \frac{1}{d} \|x\|^2, \end{cases} \quad (188)$$

where $\nu(de)$ denotes the Lévy measure. By applying the non-linear Feynman-Kac formula for jump-diffusion processes, this PIDE can be transformed into a system of FBSDEs:

$$dX_t = \frac{1}{2} \epsilon X_t dt + \tau dW_t + \int_{\mathbb{R}} e \tilde{\mu}(dt, de), \quad (189)$$

$$dY_t = \left(\lambda(\mu_\phi^2 + \sigma_\phi^2) + \tau^2 + \frac{\epsilon}{d} \|X_t\|^2 \right) dt + \tau Z_t^T dW_t + \int_{\mathbb{R}} \hat{u}_t(e) \tilde{\mu}(dt, de), \quad (190)$$

where $\tilde{\mu}(dt, de)$ is the compensated Poisson random measure. Following the Euler-Maruyama discretization, the corresponding FBSDE is given by:

$$\begin{cases} X_{n+1} = X_n + \frac{1}{2} \epsilon X_n \Delta t + \tau \Delta W_n + \sum_{i=1}^{\mu_n} z_i - \lambda \mu_\phi \Delta t, \\ Y_{n+1} = Y_n + \left(\lambda(\mu_\phi^2 + \sigma_\phi^2) + \tau^2 + \frac{\epsilon}{d} \|X_n\|^2 \right) \Delta t + \tau Z_n^T \Delta W_n + \sum_{i=1}^{\mu_n} \hat{u}_n(z_i) - \lambda \mu_\phi Z_n^T \mathbf{1} \Delta t \end{cases} \quad (191)$$

where μ_n represents the number of jump occurrences within the interval $[t_n, t_{n+1})$, which is a Poisson-distributed random variable with intensity $\lambda \Delta t$. For each jump $i = 1, \dots, \mu_n$, the jump size z_i is an independent random variable drawn from the probability distribution $\phi(e)$, modeled as a normal distribution $\mathcal{N}(\mu_\phi, \sigma_\phi^2)$. For our experiments, we set $d = 100$, $T_{\text{end}} = 1$, $\epsilon = 0.1$, $\tau = 0.1$, $\lambda = 0.01$, $\mu_\phi = 0.01$, and $\sigma_\phi = 0.01$. The true solution to this equation is $u(t, x) = \|x\|^2/d$.

We include the PIDE to test how the algorithm handles jump-diffusion processes, which are not present in standard PDEs. Although this specific case was not the primary focus during our scheme's development, we use it as a benchmark to ensure the method remains effective across various mathematical structures, including non-local operators.

F. Discussion on Efficient Higher-Order Differentiation

Throughout the main experiments, all derivatives are computed using reverse-mode automatic differentiation. This provides a simple and uniform implementation across all methods, but it does not necessarily represent the most efficient way to compute higher-order derivatives. In parallel, several recent works have proposed more efficient automatic-differentiation procedures for higher-order differential operators (Shi et al., 2024; Dangel et al., 2025).

Among these directions, Forward Laplacian provides an efficient exact procedure for computing neural-network Laplacians (Li et al., 2024). To examine the effect of such implementation-level acceleration, we additionally apply Forward Laplacian to the weighted-Laplacian computations required by FS-PINNs and Heun-BSDE. Compared with the reverse-mode automatic-differentiation implementation used in the main experiments, Forward Laplacian accelerates FS-PINNs and Heun-BSDE by approximately $3\times$. Nevertheless, both methods remain clearly slower than Un-EM-BSDE, indicating that specialized higher-order differentiation reduces but does not eliminate the computational overhead of second-order baselines.

G. Further Experiments

G.1. Ablation on M_1 and M_2

Our variance expression in (143) consists of two nonnegative terms weighted by $1/M_1 + 1/M_2$ and $1/(M_1 M_2)$. Fixing the total sampling budget $M_1 + M_2 = K$, we have $1/M_1 + 1/M_2 = K/(M_1 M_2)$; thus the variance is minimized by maximizing $M_1 M_2$, attained at $M_1 = M_2$ by AM-GM.

Table 3. Performance comparison for $M_1 = M_2 \in \{1, 2, 5, 10, 20, 50\}$ under soft and hard constraints. Values show mean RL2 error ($\times 10^{-2}$) with standard deviation ($\times 10^{-2}$) over three random seeds.

Cases	$M_1 = M_2 = 1$	$M_1 = M_2 = 2$	$M_1 = M_2 = 5$	$M_1 = M_2 = 10$	$M_1 = M_2 = 20$	$M_1 = M_2 = 50$
Soft constraint						
HJB	0.1464 ± 0.0105	0.1374 ± 0.0032	0.1348 ± 0.0153	0.1406 ± 0.0100	0.1433 ± 0.0167	0.1316 ± 0.0131
BSB	0.0805 ± 0.0049	0.0829 ± 0.0023	0.0814 ± 0.0026	0.0842 ± 0.0063	0.0869 ± 0.0023	0.0821 ± 0.0008
Hard constraint						
HJB	0.3741 ± 0.0882	0.1526 ± 0.0038	0.1444 ± 0.0275	0.1378 ± 0.0056	0.1144 ± 0.0224	0.1177 ± 0.0085
BSB	0.0205 ± 0.0001	0.0154 ± 0.0003	0.0120 ± 0.0004	0.0131 ± 0.0017	0.0118 ± 0.0015	0.0110 ± 0.0014



Figure 5. HJB($d = 100$) path prediction with error band in the soft-constraint setting.

We therefore focus on balanced configurations and report results for $M_1 = M_2 \in \{1, 2, 5, 10, 20, 50\}$ in Table 3. The performance is largely stable across these values, but $M_1 = M_2 = 5$ gives the most favorable accuracy–time balance overall. In particular, while increasing $M_1 = M_2$ monotonically increases the per-iteration sampling cost, the wall-clock training time scales moderately with $M_1 = M_2$: $M_1 = M_2 = 1$ requires $1.52\times$ the training time of EM-BSDE, $M_1 = M_2 = 2$ requires $1.55\times$, $M_1 = M_2 = 5$ requires $1.79\times$, $M_1 = M_2 = 10$ requires $2.22\times$, $M_1 = M_2 = 20$ requires $3.15\times$, and $M_1 = M_2 = 50$ requires $7.51\times$. Given that larger values provide limited additional gains in RL2 relative to their increased cost, we adopt $M_1 = M_2 = 5$ as the default setting throughout the main paper.

G.2. Additional visualizations

To further demonstrate the superior performance of our method, we provide additional visualizations of the learned trajectories for the HJB and BSB equations. The visualizations for the HJB equation under soft and hard constraint settings are presented in Figures 5 and 6, respectively. Similarly, the corresponding results for the BSB equation are illustrated in Figures 7 and 8. In Figures 7 (BSB soft constraint), Deep Shotgun has been excluded from the visualization, as the model failed to converge and was unable to produce accurate predictions.

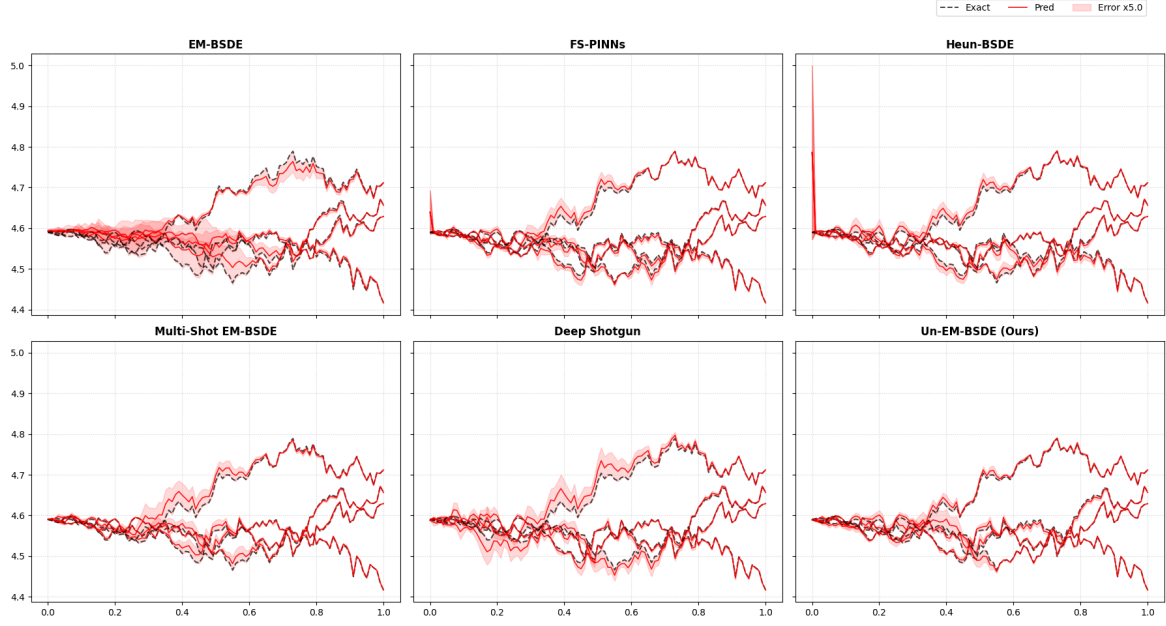


Figure 6. HJB($d = 100$) path prediction with error band in the hard-constraint setting.

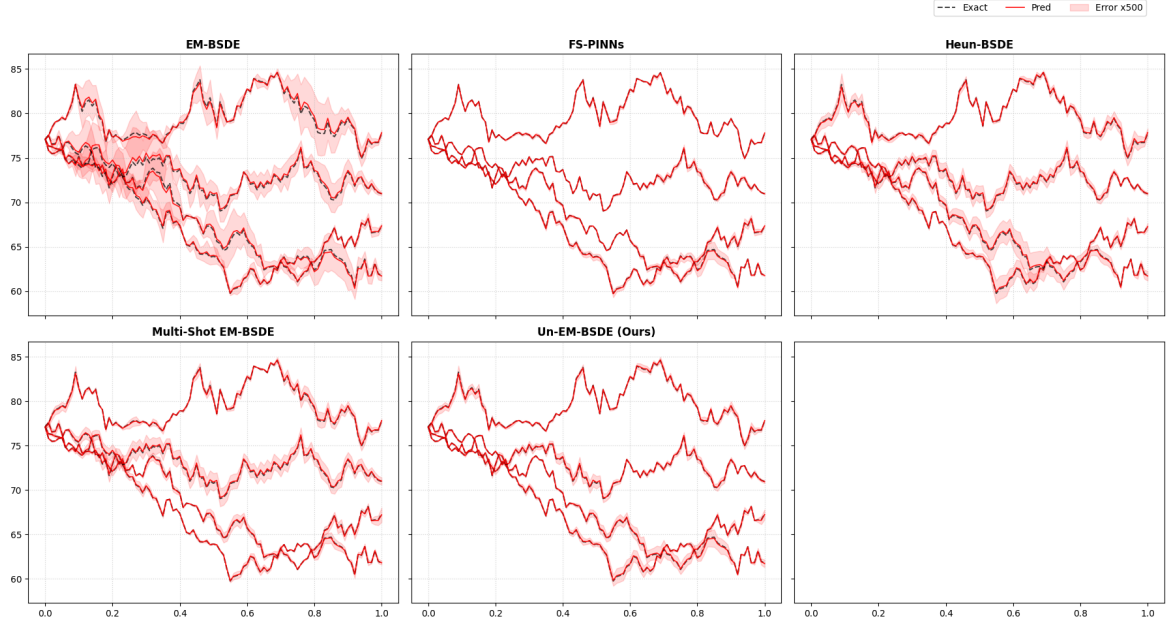


Figure 7. BSB($d = 100$) path prediction with error band in the soft-constraint setting.

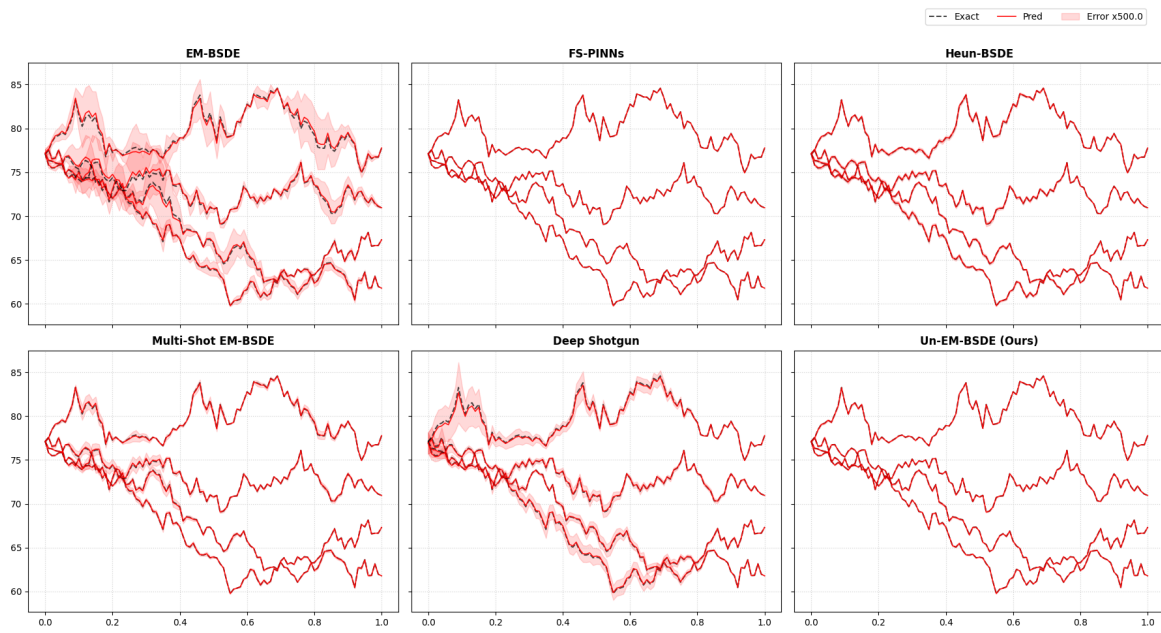


Figure 8. BSB($d = 100$) path prediction with error band in the hard-constraint setting.